

SERVICE REQUEST AND WORKFLOW AUTOMATION SYSTEM

A Project Report Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

In

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

KANURI JAGAN SATYA SAI

23225A6103

VANACHARLA VIJAYA PHANI SAI SAHITHI

22221A6155

KADIYALA YASWANTH DURGENDRA

22221A6123

PRAKKI

MOHAN SATYA SESHAGIRI

22221A6146

Under the Esteemed Guidance of

Mrs.G.Lakshmi

M.Tech.,

Assistant Professor



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE
LEARNING**

**BONAM VENKATA CHALAMAYYA ENGINEERING
COLLEGE**

(AUTONOMOUS)

(Approved by A.I.C.T.E, New Delhi & Affiliated to J.N.T.U.K, Kakinada)

(Accredited by NBA & NAAC with "A" Grade)

ODALAREVU-533210

2022 – 2026

BONAM VENKATA CHALAMAYYA ENGINEERING COLLEGE
(AUTONOMOUS)
ODALAREVU-533210

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE
LEARNING



CERTIFICATE

This is to certify that the project work entitled “**SERVICE REQUEST AND WORKFLOW AUTOMATION SYSTEM**” is being submitted for the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in **Artificial Intelligence and Machine Learning** to Bonam Venkata Chalamayya Engineering College(A), Odalarevu, is a Bonafide work done by **KANURI JAGAN SATYA SAI (23225A6103), VANACHARLA VIJAYA PHANI SAI SAHITHI (22221A6155), KADIYALA YASWANTH DURGENDRA (22221A6123), PRAKKI MOHAN SATYA SESHAGIRI(22221A6146)** under my guidance during the academic year **20252026** and it has been found suitable for acceptance according to the requirement of the University.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree.

Project Guide
KRISHNA

M.Tech.,

Assistant Professor

Head of the Department Mrs.G.Lakshmi

Dr. A. RAMA

M.Tech., Ph.D.,

Professor & HOD

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We profoundly grateful to express our deep sense of gratitude and respect towards our guide **Mrs.G.Lakshmi**, M.Tech., Assistant Professor in the CSE(AI&ML)Department, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for his excellent guidance right from selection of the project and his valuable suggestions throughout the project work. His constant encouragement and support have been the cause of our success, in completing this thesis in the college.

We are also grateful to the Project Coordinator **Mr. B. NARASIMHA RAO**, M.Tech(Ph.D.), Associate Professor in the Departments of AI&ML, CSE(AI&ML), CSE(AI&DS) Department, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for his excellent guidance right from selection of the project and her valuable suggestions throughout the project work.

We really thankful to **Dr. A. RAMA KRISHNA**, M.Tech., Ph.D., Professor and Head of the Departments of AI&ML, CSE(AI&ML), CSE(AI&DS) for providing the laboratory facilities to the fullest extent as and when required and also for giving us the opportunity to carry out the project work in the college.

We also thankful to **Dr. MAHESWAR DUTTA**, M.E., Ph.D., Principal, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for his support during and till the completion of the project.

We are also thankful to our **MANAGEMENT**, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for their support during and till the completion of the project.

We are thankful to all the **TEACHING AND NON-TEACHING STAFF** of AI&ML, CSE(AI&ML), CSE(AI&DS), Bonam Venkata Chalamayya Engineering College(A), Odalarevu, and **FRIENDS** for their direct and indirect help provided to us in completing the project.

Last but not least, we own all our success to our **FAMILY MEMBERS, CLASS MATES AND TEACHERS** from our childhood, whose vision, love and inspiration made us to reach out for these glories.

Project Associates.....

KANURI JAGAN SATYA SAI

23225A6103

VANACHARLA VIJAYA PHANI SAI SAHITHI

22221A6155

KADIYALA YASWANTH DURGENDRA

22221A6123

ABSTRACT

The Service Request and Workflow Automation System is a web-based application developed to improve organizational efficiency by automating service request processes such as submission, approval, assignment, and resolution. The system enables users to raise service requests online and track their status in real time. By automating workflows, it reduces manual effort and operational delays while enhancing service quality. The application provides role based interfaces for users, service staff, and administrators. Users can submit and monitor requests, while service staff manage assigned tasks efficiently. Administrators can configure workflows, monitor system performance, and generate reports. The system is built using modern full-stack technologies to ensure scalability, security, and reliability.

Keywords: Service Request System, Workflow Automation, Web-Based Application, Role Based Access Control, Real-Time Status Tracking, Task Assignment, Performance Monitoring, Scalability, Security.

TABLE OF CONTENT

ABSTRACT	i
TABLE OF CONTENTS	ii
CHAPTER 1 INTRODUCTION	
.....	7
1.1 Objective	7
1.2 Problem Statement	8
1.3 Software Needs	9
1.4 Hardware Needs	10
CHAPTER 2 FEASIBILITY STUDY	
.....	11
2.1 Technical Feasibility	11
2.2 Operational Feasibility	12
2.3 Economic Feasibility	12
CHAPTER 3 LITERATURE SURVEY	
.....	14
3.1 Service Request Management Systems	14
3.2 Workflow Automation Systems	15
3.3 Web Based Service Management Applications	16
3.4 Role Of Database Management In Service Systems	16
3.5 Security In Service Request And Workflow Systems.....	17
3.6 Ticket Management And Issue Tracking Systems	17
3.7 Reporting And Analytics In Service Systems	18
3.8 User Experience And Interface Design	18
CHAPTER 4 SYSTEM ANALYSIS.....	19
4.1 Existing System	19
4.2 Proposed System	20
4.3 Functional Prerequisites	20
4.3.1 User Management Module	21

4.3.2 Service Request Module	21
4.3.3 Workflow Management Module.....	22
4.3.4 Task Assignment Module	22
4.3.5 Ticket Management Module	22
4.3.6 Notification Module	22
4.3.7 Status Tracking Module	23
4.3.8 Database Management Module	23
4.3.9 Reporting And Analytics Module	23
4.3.10 Admin Control Module4.3.1.....	24
4.4 Non-Functional Prerequisites	24
4.4.1 Performance Requirements	24
4.4.2 Security Requirements	24
4.4.3 Usability Requirements	25
4.4.4 Reliability Requirements	25
4.4.5 Scalability Requirements	25
4.4.6 Maintainability Requirements	25
4.4.7 Availability Requirements	26
4.4.8 Compatibility Requirements	26
4.4.9 Data Integrity Requirements	26
CHAPTER 5	
SYSTEM DESIGN.....	27
5.1 System Architecture	27
5.1.1 Presentation Layer	28
5.1.2 Application Layer.....	29
5.1.3 Data Base Layer	29
5.2 UML Diagrams	30
5.2.1 Use Case Diagram	30
5.2.2 Class Diagram.....	32
5.2.3 Sequence Diagram	33
5.2.4 Activity Diagram	34

CHAPTER	1
IMPLEMENTATION	36
6.1 Programming Language – Java	37
6.1.1 Framework – Spring Boot	37
6.1.2 Database – MySQL	37
6.2 Architectural Design Implementation.....	38
6.2.1 Three-Tier Architecture	38
6.2.2 Advantages of the Architecture.....	38
6.3 Project Structure Implementation.....	38
6.3.1 Controller Layer Implementation	38
6.3.2 Service Layer Implementation.....	39
6.3.3 Repository and Entity Layer Implementation	39
6.4 User Authentication And Authorization	39
6.4.1 Authentication Mechanism.....	39
6.4.2 Authorization and Role Management.....	39
6.5 Service Request Management Module.....	39
6.5.1 Request Creation Process	39
6.5.2 Request Tracking and Management	40
6.6 Workflow Automation Implementation.....	40
6.6.1 Workflow Stages.....	40
6.6.2 Automated Status Transitions	40
6.7 Role-Based Dashboard Implementation.....	40
6.7.1 User Dashboard	40
6.7.2 Manager and Admin Dashboards	40
6.8 Database Implementation	40
6.8.1 Database Design	40
6.8.2 ORM And Data Access.....	41
6.9 Notification Implementation	41
6.9.1 Status Notification	41
6.10 Validation And Exception Handling.....	41

6.10.1 Input Validation	41
6.10.2 Exception Handling	41
6.11 code.....	41

CHAPTER 7 SOFTWARE ENVIRONMENT

.....
44

7.1 Overview Of The Software Environment	44
7.2 Programming Language Used	45
7.3 Framework Used	45
7.4 Frontend Technologies	46
7.5 Database Management System	46
7.6 Software Requirements	46
7.7 Security Software Environment	47
7.8 Installation And Environment Setup	47

CHAPTER 8 SYSTEM TESTING

.....**48**

8.1 INTRODUCTION TO SYSTEM TESTING	48
8.2 OBJECTIVES OF SYSTEM TESTING	48
8.3 TYPES OF TESTING PERFORMED	48
8.3.1 UNIT TESTING	49
8.3.2 INTEGRATION TESTING	49
8.3.3 SYSTEM TESTING	49
8.3.4 SECURITY TESTING	49
8.3.5 PERFORMANCE TESTING	50
8.4 TESTING STRATEGY	50
8.5 TEST CASE DESIGN	50
8.6 TESTING TOOLS USED	51
8.7 TEST RESULTS AND ANALYSIS	51

CHAPTER 9 SCREENSHOTS

.....	52	Fig 9.1 Login
Page of the Service request and workflow automation system.....	52	
Fig 9.1.1 Admin Login Page	52	
Fig 9.2: User Dashboard Home Screen.....	53	
Fig. 9.2.1 Admin Dashboard Home Screen	53	
Fig 9.2.2: Approver Dashboard Home Screen	54	
Fig. 9.3 Request page of service request and workflow automation system	54	
Fig 9.3.1: Request details page	55	
Fig. 9.4 Approver accept page	55	
Fig. 9.4.1: Request details page	56	
Fig. 9.5.1 User requests report	56	
Fig. 9.5.2 Approver Report & Analysis.....	57	
Fig. 9.6 Adding new approvers	57	
Fig 9.6.1: Adding new Users	58	
Fig. 9.7 Department management page.....	58	
CHAPTER 10	59	
10.1 CONCLUSION.....	59	
10.2 FUTURE WORK	59	
REFERENCES	60	

LIST OF FIGURES

Fig. 4.3 Data flow diagrams	21
Fig. 5.1 System Architecture	28

Fig. 5.2 UML Diagram	30
Fig 5.2.1 Use case Diagram	31
Fig. 5.2.2 Class Diagram	32
Fig. 5.2.3 Sequence Diagram	33
Fig. 5.2.4 Activity Diagram	34
Fig 9.1 Login Page	52
Fig 9.1.1 Admin Login Page	52
Fig 9.2: User Dashboard Home Screen	53
Fig. 9.2.1 Admin Dashboard Home Screen	53
Fig 9.2.2: Approver Dashboard Home Screen	54
Fig. 9.3 Request page	54
Fig 9.3.1: Request details page	55
Fig. 9.4 Approver accept page	55
Fig. 9.4.1: Request details page	56
Fig. 9.5.1 User requests report	56
Fig. 9.5.2 Approver Report & Analysis	57
Fig. 9.6 Adding new approvers	57
Fig 9.6.1: Adding new Users	58
Fig. 9.7 Department management page	58

CHAPTER 1

1. INTRODUCTION

The introduction of the Service Request and Workflow Automation System highlights the importance of automating service management processes in modern organizations to improve efficiency and reliability. In many organizations, service requests are still handled through manual methods such as emails, phone calls, and paper-based records, which often lead to delays, miscommunication, and lack of proper tracking. As the number of service requests increases, managing them manually becomes complex and time-consuming. This creates difficulties in monitoring request status, assigning tasks, and ensuring timely resolution. A centralized and automated system is therefore essential to overcome these limitations and to bring structure and transparency to service management operations.

The Service Request and Workflow Automation System provides a digital platform that automates the complete lifecycle of service requests from submission to closure. It ensures that every request is properly recorded, processed, and monitored through a predefined workflow. By implementing workflow automation, the system eliminates the dependency on human intervention and reduces the possibility of errors. The system also improves communication between users, administrators, and service personnel by maintaining clear and consistent information flow. Real-time tracking and automated notifications further enhance transparency and accountability. Overall, this system helps organizations improve productivity, reduce operational costs, and deliver faster and more reliable services.

1.1 OBJECTIVE

The main objective of the Service Request and Workflow Automation System is to develop an efficient and reliable platform that automates the management of service requests and related workflows in an organization. The system aims to replace traditional manual methods with a digital solution that ensures accuracy, transparency, and faster processing of requests. It focuses on providing a centralized environment where all service requests can be created, monitored, and managed effectively. By automating workflows, the system reduces the dependency on human intervention and minimizes the chances of errors and delays. It also aims to improve coordination

between different departments and service teams by clearly defining responsibilities at each stage of request processing. Another important objective is to enable real-time tracking so that users can easily check the status of their requests and stay informed about progress. The system seeks to enhance accountability by maintaining proper records of actions taken on every request. It also aims to improve service quality by ensuring that requests are handled systematically and within a reasonable time frame. The project intends to provide a user-friendly interface that can be easily adopted by all types of users. It also focuses on maintaining a secure and well-organized database for storing service-related information. The system is designed to support reporting and analysis for performance evaluation and decision-making. Ultimately, the objective is to create a scalable and flexible system that can adapt to future organizational requirements and contribute to overall operational efficiency.

1.2 PROBLEM STATEMENT

In many organizations, service request handling is still performed using traditional methods such as emails, phone calls, registers, and paper forms, which are inefficient and difficult to manage. These manual processes do not provide a proper structure for tracking requests, resulting in confusion and miscommunication among departments. It becomes difficult to know the current status of a request or who is responsible for resolving it. Delays often occur because there is no automatic assignment or priority management. Important requests may be overlooked or forgotten due to lack of monitoring. Data duplication and loss of information are common issues in manual systems. There is also no centralized database to store historical records for future reference. Generating reports and analyzing performance becomes a challenging task without proper documentation. The lack of transparency leads to dissatisfaction among users and affects organizational productivity. Communication between users and service teams remains weak due to unorganized workflows. Accountability is difficult to maintain as responsibilities are not clearly defined. Manual tracking consumes more time and requires additional manpower. Errors in recording and processing requests reduce service quality. There is no mechanism to ensure uniform handling of all service requests. The absence of automation leads to inefficiency and operational delays.

Therefore, there is a strong need for a centralized and automated system. The proposed Service Request and Workflow Automation System addresses these issues. It provides a structured and transparent workflow mechanism. It ensures better coordination and faster service delivery. Hence, the problem lies in the inefficiency of manual systems, which this project aims to eliminate.

1.3 SOFTWARE NEEDS

The successful implementation of the proposed system requires a reliable and efficient software environment. The following software components are essential for the development and execution of the project:

- **Operating System:** Windows 10 or above is used for development and execution of the system due to its compatibility with the required development tools and libraries.
- **Programming Language:** Java is used as the primary programming language because of its simplicity, extensive library support, and suitability for data analysis, AI, and web-based dashboards.
- **Frontend Technologies:** HTML, CSS, and JavaScript are used to design and enhance the user interface, particularly for login pages and static frontend components.
- **Dashboard Framework:** Streamlit is used to develop the interactive dashboard, as it allows rapid development of data-driven web applications with minimal effort.
- **Data Handling Libraries:** Pandas and NumPy are used for data manipulation, preprocessing, and analysis of agricultural datasets.
- **Visualization Tools:** Plotly is used for creating interactive charts, graphs, and geospatial visualizations, enabling users to easily interpret agricultural trends.
- **AI and Machine Learning Libraries:** Libraries such as TensorFlow and Scikitlearn are used for implementing the disease detection module and advisory logic.
- **Database and File Storage:** Excel and CSV files are used to store agricultural data, while GeoJSON files are used for mapping district and mandal boundaries.
- **Development Tools:** Code editors such as Visual Studio Code and Python IDEs are used for development and debugging.

These software tools collectively support data analysis, AI integration, visualization, and user interaction required by the system.

1.4 HARDWARE NEEDS

The hardware requirements for the proposed system are minimal and costeffective, making it suitable for deployment in academic and real-world environments.

The required hardware components are as follows:

- **Processor:**

Intel Core i5 or equivalent processor (minimum requirement) to handle data processing and AI computations efficiently.

- **RAM:**

minimum of 8 GB RAM is required to support smooth execution of the dashboard, data analysis, and AI modules.

- **Storage:**

At least 256 GB of hard disk or SSD storage is required to store datasets, project files, and software dependencies.

- **Display Device:** standard monitor or laptop display with sufficient resolution to view dashboards and visualizations clearly.

- **Input Devices:**

Keyboard and mouse for interacting with the system.

- **Internet Connectivity:**

Internet access is required for software installation, library updates, and future enhancements such as real-time weather data integration.

The hardware requirements are intentionally kept minimal to ensure that the system can be easily adopted and scaled without significant investment.

CHAPTER 2

2. FEASIBILITY STUDY

The feasibility study is an important phase in the development of the Service Request and Workflow Automation System, as it helps to determine whether the proposed system can be practically implemented and will be beneficial to the organization. This study evaluates the project from different perspectives such as technical, economic, and operational feasibility. It ensures that the system can be developed using the available resources and that it will function efficiently in a real organizational environment. The feasibility study also helps in identifying possible risks and limitations at an early stage so that appropriate measures can be taken. By conducting this analysis, it becomes easier to decide whether the project is worth implementing and whether it will meet organizational requirements effectively. This chapter analyzes the feasibility of the proposed system under the following aspects:

- Technical Feasibility
- Operational Feasibility
- Economic Feasibility

2.1 TECHNICAL FEASIBILITY

Technical feasibility refers to the availability of technology, hardware, software, and technical expertise required to develop and implement the proposed system. The Service Request and Workflow Automation System can be developed using widely available web technologies such as HTML, CSS, and JavaScript for the front end, along with backend technologies like Python, Java, or PHP. Databases such as MySQL or MongoDB are suitable for storing and managing service request data efficiently. These technologies are well established, reliable, and easy to maintain.

The existing hardware infrastructure in most organizations is sufficient to support the development and deployment of the system. No special or high-end hardware is required, which makes the project technically feasible. The system can run on standard computers and servers with basic configurations. Moreover, developers and technical staff are generally familiar with the technologies used, which reduces the complexity of development and maintenance.

The system design is modular, making it easier to update, enhance, and scale in the future. Any new features or workflow changes can be easily integrated without

affecting the existing structure. Since the technologies used are stable and supported by large developer communities, troubleshooting and technical support are easily available. Therefore, from a technical point of view, the Service Request and Workflow Automation System is completely feasible and practical to implement.

2.2 OPERATIONAL FEASIBILITY

Operational feasibility focuses on how well the proposed system will be accepted and used by its intended users. The Service Request and Workflow Automation System is designed to be user-friendly, with a simple and intuitive interface that makes it easy for users to submit, track, and manage service requests. Since the system follows a clear and structured workflow, users can quickly understand how it operates. Minimal training is required for users to adapt to the system, which increases its acceptance within the organization. The automated notifications and real-time tracking features improve communication between users, administrators, and service personnel. This reduces confusion and ensures that everyone is informed about the status of service requests.

The system reduces the workload of administrative staff by automating repetitive tasks and maintaining accurate digital records. It improves transparency and accountability by clearly defining responsibilities at each stage of the workflow. As a result, users are more likely to trust and rely on the system for managing service requests. Therefore, the proposed system is operationally feasible and suitable for real-world organizational use.

2.3 ECONOMIC FEASIBILITY

Economic feasibility deals with the cost-effectiveness of the proposed system and determines whether the benefits gained from the system justify the investment made. The Service Request and Workflow Automation System is economically feasible because it uses mostly open-source tools and technologies, which significantly reduce development and licensing costs. The required software platforms are either freely available or available at minimal cost.

The implementation of this system reduces the dependency on manual processes, paperwork, and excessive manpower. This leads to a reduction in operational costs over time. Automated handling of service requests minimizes delays and errors, which

in turn improves productivity and efficiency. The organization can save money by optimizing resource utilization and reducing administrative overhead.

Although there may be some initial costs involved in development and deployment, these costs are relatively low compared to the long-term benefits. The system improves service quality, speeds up request processing, and reduces recurring expenses related to manual management. Hence, the financial investment in developing this project is justified, and the project is economically feasible.

CHAPTER 3

3.LITERATURE SURVEY

The literature survey provides an overview of existing research, tools, and technologies related to service request management and workflow automation systems. It helps in understanding how similar systems are designed and implemented, and what improvements can be introduced in the proposed system. By studying previous works,

we can identify the strengths and limitations of existing approaches and use this knowledge to build a more efficient and reliable solution. The survey also highlights the importance of automation in improving organizational efficiency and service quality. Many researchers have emphasized that manual handling of service requests leads to inefficiency, lack of transparency, and delayed service delivery. Traditional systems depend heavily on human intervention, which increases the chances of errors and data inconsistency. Modern workflow automation systems aim to replace these manual processes with digital solutions that ensure systematic processing and better coordination between departments. Workflow automation allows organizations to define a sequence of tasks and approvals that must be followed for each service request, thereby ensuring uniformity and accountability.

3.1 SERVICE REQUEST MANAGEMENT SYSTEMS

Service Request Management Systems are designed to provide a structured and organized approach to handle various service-related activities within an organization. These systems act as a centralized platform where users can submit their service requests in a standardized format, ensuring that all necessary details are captured accurately. Literature shows that centralized request management eliminates the confusion created by scattered communication methods such as emails, phone calls, and paper forms. It improves coordination among departments by maintaining all service-related information in a single repository. Service request systems allow administrators to categorize requests based on type, priority, and urgency, which helps in better allocation of resources and faster problem resolution. They also maintain a complete history of requests, making it easier to analyze past performance and identify recurring issues. Researchers emphasize that these systems increase transparency by providing real-time updates on the status of requests. Users are no longer dependent on manual follow-ups to know the progress of their requests, which improves satisfaction and trust. Service request management systems also enhance accountability by recording every action taken on a request along with time and user details. This makes it easier to identify delays and responsible personnel. Automation reduces human errors such as data duplication and miscommunication. These systems support better documentation and improve service quality by ensuring that every request is handled in a systematic

manner. However, many traditional service request systems focus only on request registration and tracking, without integrating workflow automation. This limits their efficiency in handling complex organizational processes. Therefore, modern systems emphasize the need for combining service request management with workflow automation to achieve complete process automation.

3.2 WORKFLOW AUTOMATION SYSTEMS

Workflow Automation Systems are developed to automate and manage business processes by defining a clear and structured sequence of steps that a task must follow from initiation to completion. These systems ensure that every activity is performed in a systematic manner without unnecessary manual intervention. Literature highlights that workflow automation significantly reduces processing time by automatically routing tasks to the appropriate individuals or departments. It removes the dependency on manual task assignment, which often causes delays and miscommunication. Workflow automation improves efficiency by ensuring that tasks are executed in the correct order and within predefined timelines.

Researchers point out that automated workflows enhance accountability by clearly defining responsibilities at each stage of the process. Every action performed on a task is recorded, which helps in monitoring performance and identifying delays. Workflow systems also increase transparency, as stakeholders can track the progress of tasks in real time. They improve consistency in operations by ensuring that the same rules and procedures are followed for every request. Automation minimizes errors that commonly occur due to human negligence or misunderstanding.

Workflow automation systems also support better collaboration between departments by maintaining a smooth flow of information. They reduce dependency on emails and verbal communication, which are often unreliable and difficult to track. These systems provide alerts and notifications that keep users informed about task updates and deadlines. Studies indicate that workflow automation increases productivity by reducing repetitive tasks and freeing employees to focus on more critical activities.

3.3 WEB-BASED SERVICE MANAGEMENT APPLICATIONS

Web-based service management applications have become highly popular due to their accessibility, flexibility, and ease of use. These applications allow users to access

the system from any location using a web browser, which makes them highly convenient for organizations with distributed teams. Literature shows that web-based systems reduce dependency on physical infrastructure and eliminate the need for installing software on individual machines. They provide a centralized platform where all service-related data is stored and managed efficiently. Web-based systems support realtime communication between users and service providers, improving coordination and response time.

Researchers highlight that web applications are easier to maintain and update, as changes can be implemented directly on the server without affecting users. They improve scalability, allowing organizations to handle increasing numbers of service requests without major changes to infrastructure. Web-based systems also promote collaboration by enabling multiple users to work on the same platform simultaneously. They support integration with other systems such as databases, notification services, and analytics tools.

3.4 ROLE OF DATABASE MANAGEMENT IN SERVICE SYSTEMS

Database management plays a critical role in the performance and reliability of service request and workflow automation systems. Literature emphasizes that databases act as the backbone of any automation system by storing all service-related data in a structured format. They ensure data consistency, integrity, and accuracy throughout the system. Centralized databases allow easy access to historical records, which helps in auditing and performance evaluation.

Researchers explain that proper database design improves system efficiency and reduces data redundancy. Well-structured tables and relationships help in faster data retrieval and processing. Databases support reporting and analytics by providing organized data for generating insights. They also help in maintaining user information, service logs, and workflow records.

Literature highlights the importance of backup and recovery mechanisms to ensure data safety in case of system failures. Indexing and optimization techniques improve query performance. Both relational and non-relational databases are widely used depending on system requirements. Database management also supports multi-user access and

concurrent operations. It ensures secure storage of sensitive information. Without proper database management, service automation systems become unreliable and inefficient.

3.5 SECURITY IN SERVICE REQUEST AND WORKFLOW SYSTEMS

Security is one of the most critical aspects of service automation systems, as they deal with sensitive organizational and user data. Literature emphasizes that strong authentication mechanisms are required to ensure that only authorized users can access the system. Authorization controls define what actions each user can perform based on their role

Security also includes protection against cyber threats such as hacking, data breaches, and malware attacks. Firewalls and secure servers are recommended to protect the application. Regular system updates and patches reduce vulnerabilities. Data backups ensure recovery in case of security failures. Literature shows that a secure system ensures reliability, data privacy, and compliance with organizational policies.

3.6 TICKET MANAGEMENT AND ISSUE TRACKING SYSTEMS

Ticket management systems are widely used to organize and track service requests systematically. Literature shows that converting service requests into tickets provides a unique identity for each request, which simplifies monitoring and resolution. Ticketing systems improve accountability by ensuring that no request is overlooked.

Ticket management improves communication between users and service providers by providing a structured communication channel. It reduces dependency on emails and phone calls. However, literature states that many ticketing systems lack workflow integration. Combining ticket management with workflow automation creates a more powerful and efficient service management platform.

3.7 REPORTING AND ANALYTICS IN SERVICE SYSTEMS

Reporting and analytics are essential for evaluating system performance and service quality. Literature shows that reports help administrators analyze response times, resolution rates, and workload distribution. These reports support decision-making and resource planning.

Data visualization improves understanding of performance trends. Reports increase transparency and accountability. Analytics support continuous improvement by identifying areas that require enhancement. Literature emphasizes that reporting and analytics transform raw data into meaningful information for organizational growth

3.8 USER EXPERIENCE AND INTERFACE DESIGN

User experience is a major factor influencing the success of automation systems. Literature shows that simple and intuitive interfaces increase user adoption. A well-designed interface reduces training requirements and improves productivity. Responsive design ensures compatibility with different devices. Visual feedback guides users and reduces errors. Literature states that poor interface design leads to resistance and underutilization. Therefore, user-friendly design is essential for effective service automation.

CHAPTER 4

4. SYSTEM ANALYSIS

System analysis is the most important phase in software development, as it involves a detailed study of the existing system and identification of its drawbacks. It also helps in understanding the requirements of the new system. In the current scenario, most organizations still follow manual or semi-automated methods for

handling service requests. These methods depend on emails, phone calls, or written forms, which are not well organized and are difficult to manage. Due to the lack of a centralized platform, tracking the status of service requests becomes complicated and time consuming. There is no proper mechanism to ensure accountability and transparency in the service handling process.

The existing approach often results in delays, data loss, miscommunication, and inefficient resource utilization. Since records are maintained in different formats and locations, retrieving historical information becomes difficult. Reporting and performance evaluation are almost impossible without structured data. Manual processes also increase the chances of human errors, duplication of work, and inconsistency in service handling.

System analysis helps in identifying these issues and provides a clear understanding of what improvements are required. The proposed Service Request and Workflow Automation System is designed to overcome these challenges by introducing a centralized, automated, and transparent platform. It focuses on improving efficiency, accuracy, and accountability while reducing manual workload. This chapter discusses the existing system, its limitations, and the advantages of the proposed system.

4.1 EXISTING SYSTEM

The existing system for handling service requests is mostly manual in nature. Users raise their requests through phone calls, emails, or by submitting written applications. These requests are then processed manually by staff members. Since there is no centralized database, information is scattered and difficult to manage. Requests are often missed or delayed due to improper tracking mechanisms.

There is no defined workflow that specifies how a service request should move from one stage to another. Task assignment is done manually, which leads to inefficiency and confusion. Communication between departments is unstructured, and users are not informed about the real-time status of their requests.

The existing system does not support reporting and analytics. Administrators cannot measure performance or identify problem areas. It is difficult to maintain historical records. Manual documentation consumes more time and effort. Overall, the existing system is unreliable, slow, and inefficient for modern organizational needs.

due to repeated follow-ups and inefficient task handling.

4.2 PROPOSED SYSTEM

The proposed **Service Request and Workflow Automation System** introduces a fully automated and centralized approach to managing service requests. It provides a webbased platform where users can submit service requests easily. Each request is processed through a predefined workflow that ensures systematic handling from initiation to completion.

The system automatically assigns requests to the appropriate department or personnel based on defined rules. Real-time tracking allows users and administrators to monitor request progress at every stage. Automated notifications keep users informed about updates, approvals, and request completion.

All service-related data is stored in a centralized database, ensuring data consistency, security, and easy retrieval. The system also supports reporting and analytics, enabling administrators to evaluate performance and identify bottlenecks. Automation reduces manual workload, improves accuracy, and enhances overall efficiency.

4.3 FUNCTIONAL PREREQUISITES

Functional prerequisites define the essential functions and capabilities that the system must provide in order to operate effectively. These prerequisites describe what the **Service Request and Workflow Automation System** should do to meet user and organizational requirements. They focus on system behavior, features, and interactions rather than technical implementation details.

The User Management Module is responsible for handling all user-related operations in the system. It ensures secure access through authentication and controls user permissions using role-based authorization.

- ### 4.3.2 SERVICE REQUEST MODULE

- Enables service request submission
- Stores request information securely
- Generates unique request IDs
- Validates request data

- Initiates workflow automation

4.3.3 WORKFLOW MANAGEMENT MODULE

The Workflow Management Module defines and controls the sequence of steps involved in processing service requests. It ensures that each request follows a structured and predefined flow.

- Defines workflow stages
- Automates request routing
- Ensures process consistency
- Controls approvals and processing
- Reduces manual intervention

4.3.4 TASK ASSIGNMENT MODULE

The Task Assignment Module automatically assigns service requests to the appropriate staff or department. It ensures efficient workload distribution and faster service response.

- Assigns tasks based on roles
- Balances staff workload
- Reduces assignment delays
- Improves response time
- Tracks task responsibility

4.3.5 TICKET MANAGEMENT MODULE

The Ticket Management Module converts service requests into trackable tickets. It helps in monitoring, prioritizing, and resolving service requests systematically.

- Generates unique ticket numbers
- Tracks ticket lifecycle
- Supports request prioritization
- Maintains ticket history
- Improves accountability

4.3.6 NOTIFICATION MODULE

The Notification Module provides automated alerts to users and staff regarding request updates. It ensures timely and effective communication within the system.

-
-
-
-
- Sends request status notifications
 - Supports email and system alerts
 - Reduces manual follow-ups
 - Keeps users informed
 - Enhances coordination

4.3.7 STATUS TRACKING MODULE

The Status Tracking Module allows real-time monitoring of service requests. It provides transparency and visibility into request progress.

- Displays real-time request status
- Enables progress monitoring
- Improves transparency
- Reduces user uncertainty
- Enhances satisfaction

4.3.8 DATABASE MANAGEMENT MODULE

The Database Management Module stores all system data in a structured and secure manner. It ensures data integrity, consistency, and availability.

- Maintains centralized data storage
- Stores user and request data
- Supports data retrieval and updates
- Ensures data integrity
- Supports reporting functions

4.3.9 REPORTING AND ANALYTICS MODULE

The Reporting and Analytics Module generates reports to evaluate system performance. It helps administrators make informed decisions based on system data.

- Generates service reports
- Analyzes request trends
- Measures performance metrics

-
-
-
-
- Identifies bottlenecks
- Supports decision-making

4.3.10 ADMIN CONTROL MODULE

The Admin Control Module provides administrative control over the system. It allows configuration, monitoring, and management of system operations.

Manages users and roles

Configures workflows

Monitors system activities

Controls system settings

- Ensures smooth system operation

4.4 NON-FUNCTIONAL PREREQUISITES

Non-functional requirements describe the quality attributes and constraints of the Service Request and Workflow Automation System. These requirements focus on how the system performs rather than what functions it performs. They play a crucial role in ensuring reliability, usability, security, and overall system effectiveness.

4.4.1 PERFORMANCE REQUIREMENTS

The system should respond quickly to user requests and operations. Service request submission, status updates, and report generation must be performed within acceptable response times to ensure smooth user experience.

- Supports fast request processing
- Handles multiple users simultaneously
- Ensures minimal system response time
- Maintains performance under heavy load
- Provides smooth workflow execution

-
-
-
-

4.4.2 SECURITY REQUIREMENTS

Security is essential to protect sensitive service and user data. The system must restrict unauthorized access and ensure data privacy through proper security mechanisms.

- Secure user authentication
- Role-based authorization
- Protection against unauthorized access
- Secure data storage
- Maintains user confidentiality

4.4.3 USABILITY REQUIREMENTS

The system should be easy to use and understand by all types of users. A simple and intuitive interface reduces training requirements and improves user acceptance.

User-friendly interface

Simple navigation

Minimal training required

Clear instructions and messages

- Easy access to system features

4.4.4 RELIABILITY REQUIREMENTS

The system must operate consistently without failures. It should ensure continuous availability and accurate processing of service requests.

- Reliable request handling
- Consistent system operation
- Accurate data processing
- Minimal system downtime
- Error-free workflow execution

-
-
-
-

4.4.5 SCALABILITY REQUIREMENTS

The system should support future growth without performance degradation. It must handle increasing users and service requests efficiently.

- Supports increased user load
- Handles large number of requests
- Easy system expansion
- Flexible architecture

4.4.6 MAINTAINABILITY REQUIREMENTS

The system should be easy to maintain and update. Changes and enhancements must be implemented without affecting existing functionality.

- Easy bug fixing
- Simple system updates
- Modular system design
- Easy troubleshooting
- Supports future enhancements

4.4.7 AVAILABILITY REQUIREMENTS

The system should be available whenever users need to access it. High availability improves reliability and user trust.

24/7 system availability

Continuous user access

Minimal downtime

Stable system performance

- Reliable service access

4.4.8 COMPATIBILITY REQUIREMENTS

The system should work across different devices and browsers. Compatibility ensures accessibility for all users.

- Works on multiple browsers
- Device-independent access

-
-
-
-
- Supports different operating systems
- Responsive design
- Easy web accessibility

4.4.9 DATA INTEGRITY REQUIREMENTS

The system must ensure accuracy and consistency of stored data. Proper validation and controls must be implemented.

- Accurate data storage
- Prevents data duplication
- Maintains data consistency
- Validates input data

CHAPTER 5

5. SYSTEM DESIGN

System design is a critical phase in the software development life cycle that translates system requirements into a structured and logical design. This chapter explains how the **Service Request and Workflow Automation System** is architected and how its components interact with each other. The design focuses on modularity, scalability, security, and ease of maintenance.

The system is designed using a modular approach, where each module performs a specific function. This improves clarity, simplifies implementation, and allows future enhancements without affecting the entire system. The design ensures smooth data flow between modules and efficient handling of service requests.

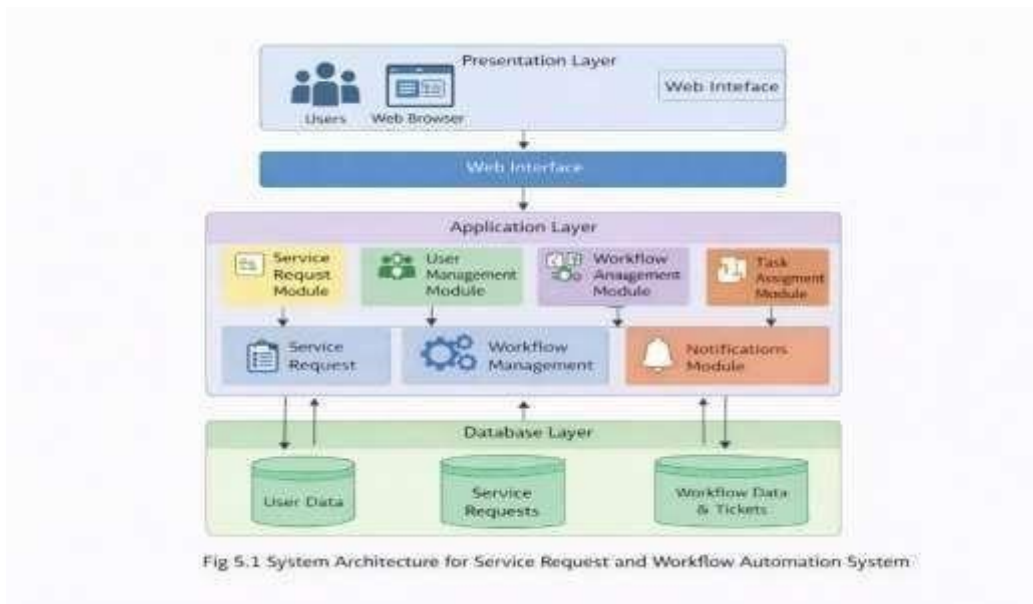
5.1 SYSTEM ARCHITECTURE

The proposed system follows a **web-based client-server architecture**. In this architecture, users interact with the system through a web interface, while the backend server processes requests and manages workflows. The architecture is divided into three major layers: Presentation Layer, Application Layer, and Database Layer. This layered structure improves system performance, security, and maintainability.

The architecture ensures that user requests are processed efficiently and that all service-related data is stored securely. Each layer performs a specific function and communicates with other layers in a controlled manner.

The architecture of the **Service Request and Workflow Automation System** consists of the following major layers:

- Presentation Layer
- Application Layer
- Database Layer



5.1.1 PRESENTATION LAYER

The presentation layer is the topmost layer of the system and acts as an interface between users and the application. It allows users to interact with the system by submitting service requests, tracking status, and viewing reports. This layer ensures ease of use and smooth interaction.

- User login and authentication
- Service request submission
- Request status tracking
- Dashboard and report viewing
- User-friendly interface

The presentation layer validates user inputs and forwards them to the application layer for processing. It improves user experience by providing simple navigation and clear system responses.

5.1.2 APPLICATION LAYER

The application layer is the core of the system where all business logic and workflow processing are performed. It handles service request processing, workflow automation, task assignment, notifications, and report generation. This layer ensures that all operations follow predefined business rules.

- Service request processing
- Workflow management
- Task assignment and escalation
- Notification handling

- Report generation

The application layer acts as a bridge between the user interface and the database. It processes requests efficiently and ensures automation, accuracy, and accountability within the system.

5.1.3 DATA BASE LAYER

The database layer is responsible for storing and managing all system-related data securely. It maintains user details, service requests, workflow logs, task records, and reports. This layer ensures data integrity and consistency.

- User data storage
- Service request records
- Workflow and task logs
- Report data storage
- Secure data management

The database layer supports data retrieval and updates requested by the application layer. It plays a crucial role in maintaining system reliability and enabling report generation

Dept. of AIML, BVCEC, Odalarevu

5.2 UML DIAGRAMS

Unified Modeling Language (UML) diagrams are used to visually represent the structure and behavior of the system. UML diagrams help in understanding system functionality, data flow, and interaction between components. For the **Service request and workflow automation system**, UML diagrams are used to model user interaction, system structure, and processing flow.

The following UML diagrams are used in the system design:

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Activity Diagram

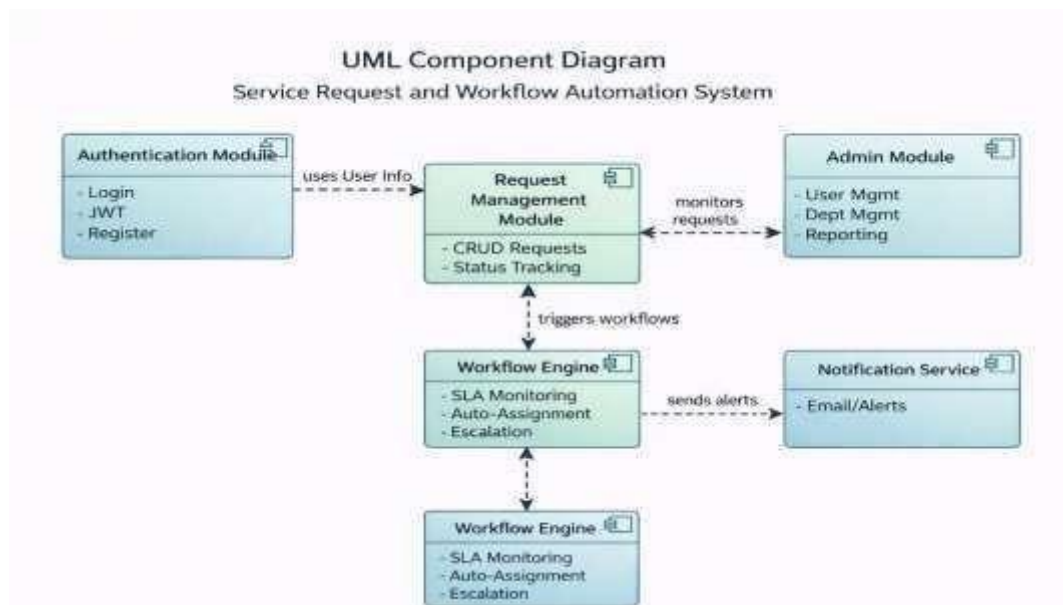


Fig 5.2 UML Diagram of the Service request and workflow automation System

5.2.1 USE CASE DIAGRAM

The Use Case Diagram illustrates system functionality from the user's perspective. It defines the interactions between different actors and the system within a clear system boundary. The diagram identifies roles such as Employee, Approver, and Administrator

and maps their permitted operations. This ensures role-based access control and functional completeness of the system.

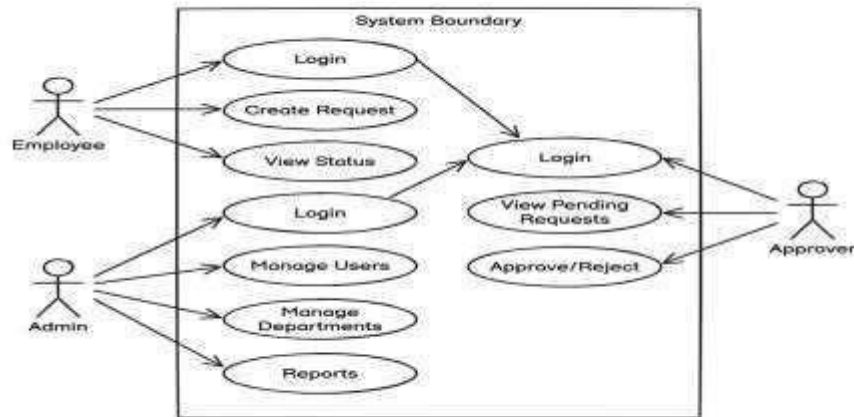


Fig 5.2.1 Use case Diagram of the Service request and workflow automation System

- Actor identification
- System boundary definition
- User request creation
- Approval and rejection actions
- Status tracking functionality
- Administrative operations
- Report generation

Explanation

The Use Case Diagram outlines system functionality from a user-centric perspective. It clearly identifies system actors and their permitted interactions within the defined system boundary. Employees initiate service requests and monitor progress, approvers evaluate and authorize requests, and administrators manage users, departments, and reports. This diagram ensures requirement completeness, role clarity, and access control enforcement.

5.2.2 CLASS DIAGRAM

The Class Diagram represents the static structure of the system by defining classes, their attributes, and relationships. It provides a blueprint of how data and functionalities are organized within the system.

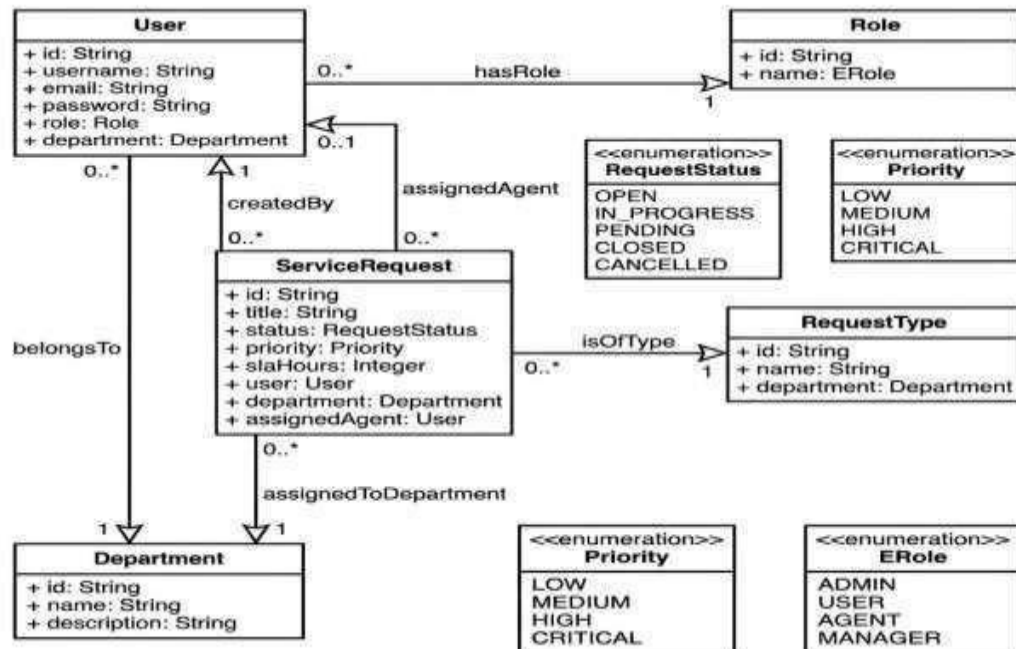


Fig. 5.2.2 Class Diagram of the Service request and workflow automation system

Explanation

The Class Diagram shows how different system components interact with each other. Dashboard class acts as the central controller, communicating with the DataManager, AnalyticsEngine, AdvisoryEngine, and MapVisualizer. The modular design ensures separation of concerns and improves maintainability.

5.2.3 SEQUENCE DIAGRAM

The Sequence Diagram illustrates the flow of control and messages exchanged between system components over time. It helps in understanding the execution sequence of system operations.

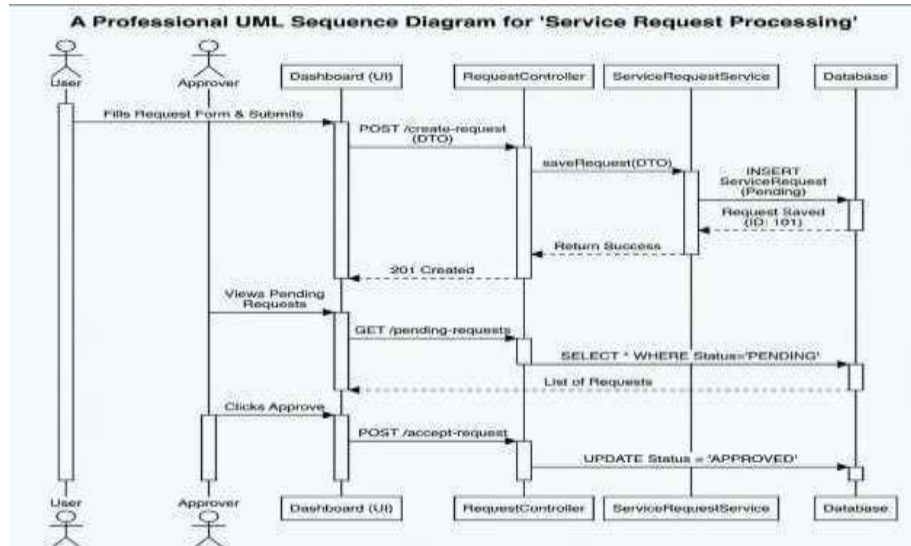


Fig. 5.2.3 Sequence Diagram of the Service request and workflow automation System **Sequence Flow**:

1. The user logs into the system and submits a service request through the dashboard.
2. The user interface sends the request data to the Request Controller.
3. The Request Controller forwards the data to the Service Request Service.
4. The service validates the request and stores it in the database with **PENDING** status.
5. The database returns a success response with a generated request ID.
6. The approver logs in and requests the list of pending service requests.
7. The system retrieves pending requests from the database and displays them.
8. The approver selects a request and approves it.
9. The system updates the request status to **APPROVED** in the database.
10. The workflow triggers agent assignment and notifications, completing the process.

Explanation

The Sequence Diagram provides a detailed temporal view of interactions between system components during service request processing. It emphasizes the communication flow among the user interface, controller layer, service layer, and

database. The diagram illustrates how request data is transferred using data transfer objects (DTOs), processed by the service layer, and persisted in the database. Approval actions trigger database updates and system responses, ensuring real-time status synchronization

5.2.4 ACTIVITY DIAGRAM

The Activity Diagram represents the workflow of activities performed within the system. It emphasizes the decision points and parallel processing steps.

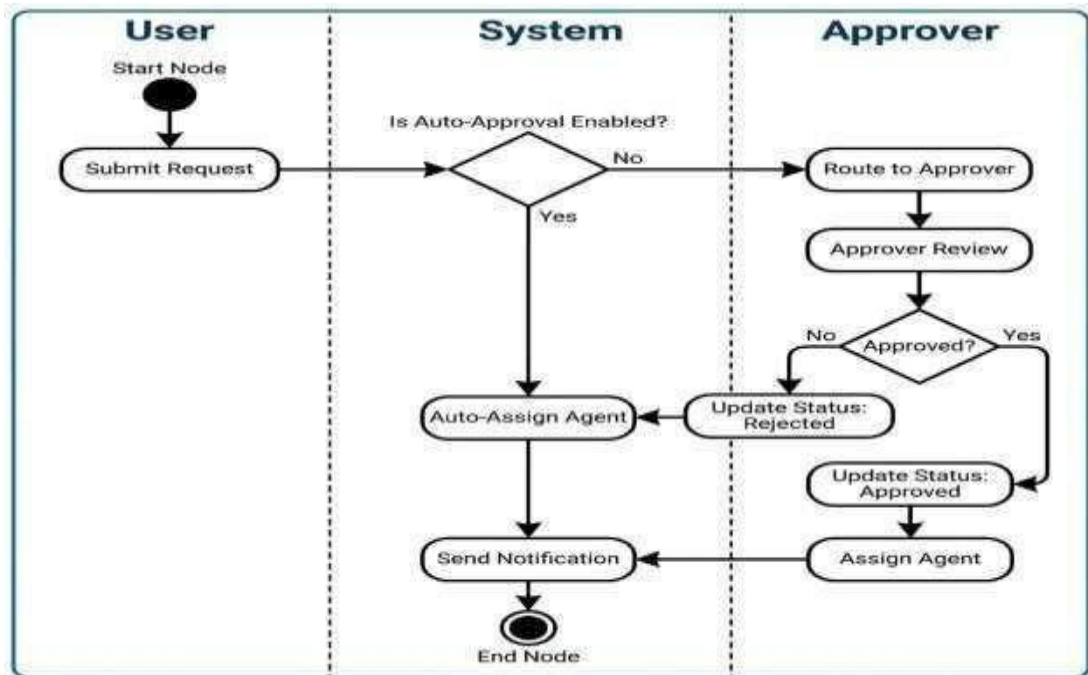


Fig. 5.2.4 Activity Diagram Showing User Interaction Flow in the Proposed System

Workflow Description

- Start
- User Login
- Validate Credentials
- Access Dashboard
- Create Service Request
- Enter Request Details
- Submit Request
- Check Auto-Approval Rule

- If Auto-Approved → Assign Agent
- Else → Forward to Approver
- Approver Reviews Request
- Approve / Reject Decision
- Update Request Status
- Notify User and Agent
- Track Request Progress
- Close Request
- User Logout • End

Explanation

The Activity Diagram models the dynamic behavior of the Service Request and Workflow Automation System by representing the complete lifecycle of a service request. It clearly defines the sequence of activities, decision nodes, and role responsibilities across user, system, and approver swim lanes.

CHAPTER 6

6. IMPLEMENTATION

The implementation of the **Service Request and Workflow Automation System** involves transforming the system design and UML models into a fully functional

software application. This phase focuses on developing system modules, integrating components, and ensuring that all functional requirements are met effectively. The implementation follows a modular and layered approach to enhance scalability, security, and maintainability.

The system is developed using a client–server architecture, where the frontend provides an interactive user interface and the backend handles business logic, workflow processing, and data management. Role-based access control is implemented to ensure that users such as employees, approvers, agents, and administrators can access only permitted functionalities.

6.1 IMPLEMENTATION ENVIRONMENT

The implementation environment defines the software and hardware setup required to develop and execute the system. Choosing an appropriate environment is essential for achieving reliability, performance, and scalability.

6.1.1 Programming Language – Java

Java is used as the core programming language for implementing the Service Request and Workflow Automation System. It provides platform independence, strong security features, and object-oriented concepts that support modular development. Java enables the development of robust backend logic required for workflow automation and request processing. Its extensive libraries and community support make it suitable for enterprise-level applications.

6.1.2 Framework – Spring Boot

Spring Boot is used as the primary framework to simplify application development. It reduces configuration complexity by providing autoconfiguration and embedded server support. Spring Boot allows easy integration of components such as Spring MVC, Spring Security, and Spring Data JPA. This framework helps in rapid development, easy maintenance, and efficient deployment of the application.

6.1.3 Database – MySQL

MySQL is used as the relational database management system for storing application data. It stores user information, service requests, workflow status, and approval history.

MySQL ensures data consistency, integrity, and security. It supports efficient query processing and reliable data storage, which is essential for workflow-based applications.

6.2 ARCHITECTURAL DESIGN IMPLEMENTATION

The system architecture defines how different components interact with each other. A structured architecture ensures scalability, maintainability, and efficient processing of service requests.

6.2.1 Three-Tier Architecture

The system follows a three-tier architecture consisting of presentation layer, business logic layer, and data access layer. The presentation layer handles user interaction through web interfaces. The business logic layer processes service requests and applies workflow rules. The data access layer manages database communication using repositories. This separation of layers improves system flexibility and simplifies maintenance.

6.2.2 Advantages of the Architecture

The three-tier architecture improves modularity by separating concerns. It allows independent modification of layers without affecting the entire system. It enhances security by restricting direct database access. This architecture also supports future enhancements and scalability.

6.3 PROJECT STRUCTURE IMPLEMENTATION

The project structure is designed to organize source code efficiently and improve readability. A well-defined structure helps in managing large applications.

6.3.1 Controller Layer Implementation

Controller classes handle incoming HTTP requests from users. They act as an interface between the frontend and backend. Controllers receive user input, forward requests to service classes, and return responses to the user interface. This layer ensures smooth interaction between users and the system.

6.3.2 Service Layer Implementation

The service layer contains the core business logic of the system. It implements workflow automation rules, request processing logic, and validation mechanisms. Service classes coordinate between controllers and repositories. This layer ensures that business rules are applied consistently throughout the application.

6.3.3 Repository and Entity Layer Implementation

Repository classes manage database operations using Spring Data JPA. Entity classes represent database tables and define relationships between data. This layer abstracts database interaction and simplifies CRUD operations. It improves code maintainability and data consistency.

6.4 USER AUTHENTICATION AND AUTHORIZATION

Security is a critical aspect of the system to protect sensitive data and restrict access.

6.4.1 Authentication Mechanism

Authentication verifies the identity of users accessing the system. The application provides login and logout functionality. User credentials are validated against encrypted passwords stored in the database. This mechanism ensures that only valid users can access the system.

6.4.2 Authorization and Role Management

Authorization controls user access based on roles such as Admin, Manager, and User. Each role has specific permissions. Spring Security enforces access control rules. This prevents unauthorized access to restricted functionalities and ensures data security.

6.5 SERVICE REQUEST MANAGEMENT MODULE

The service request module is responsible for handling request creation and management.

6.5.1 Request Creation Process

Users submit service requests by entering required details through the user interface. The system validates input data and generates a unique request ID. The request is stored in the database with an initial status of Pending. This process ensures standardized request submission.

6.5.2 Request Tracking and Management

Users can view the current status and history of their service requests. Managers can review and take action on pending requests. This functionality improves transparency and accountability.

6.6 WORKFLOW AUTOMATION IMPLEMENTATION

Workflow automation manages the entire life cycle of service requests.

6.6.1 Workflow Stages

The workflow consists of multiple stages including submission, approval, processing, and completion. Each stage represents a specific state of the request. Status changes occur automatically based on user actions and predefined rules.

6.6.2 Automated Status Transitions

The system automatically updates request status as it moves through the workflow. This reduces manual intervention and processing delays. Automated transitions ensure consistency and accuracy in request handling.

6.7 ROLE-BASED DASHBOARD IMPLEMENTATION

Dashboards provide role-specific views of system data.

6.7.1 User Dashboard

The user dashboard allows users to submit service requests and monitor their progress. It provides an overview of request status and history.

6.7.2 Manager and Admin Dashboards

Manager dashboards allow approval or rejection of requests. Admin dashboards provide system management features such as user and service management. These dashboards improve usability and operational efficiency.

6.8 DATABASE IMPLEMENTATION

The database implementation ensures secure and efficient data storage.

6.8.1 Database Design

Database tables are designed to store users, roles, service requests, workflow status, and approval history. Relationships are established to maintain data integrity.

Dept. of AIML, BVCEC, Odalarevu

6.8.2 ORM and Data Access

Spring Data JPA is used for object-relational mapping. It simplifies database interaction and reduces boilerplate code. This approach improves development speed and maintainability.

6.9 NOTIFICATION IMPLEMENTATION

The notification module improves communication between system users.

6.9.1 Status Notification

Users receive notifications when request status changes. Managers are notified about pending approvals. This ensures timely action and transparency.

6.10 VALIDATION AND EXCEPTION HANDLING

Validation and exception handling improve system reliability.

6.10.1 Input Validation

Server-side validation ensures that only valid data is processed. It prevents incorrect or incomplete submissions.

6.10.2 Exception Handling

Custom exception handling manages runtime errors and displays meaningful error messages. This prevents system crashes and improves user experience.

6.11 Code :

```
package com.softwarecompany.serviceportal.controllers; import
com.softwarecompany.serviceportal.models.User; import
com.softwarecompany.serviceportal.services.AdminService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize; import
org.springframework.web.bind.annotation.*;

import java.util.List;
@CrossOrigin(origins = "*", maxAge = 3600)
@RestController
```

```
@RequestMapping("/api/admin/users")
@PreAuthorize("hasRole('ADMIN')")
public class AdminUserController {

    @Autowired
    AdminService adminService;

    @GetMapping                                public
    List<User> getAllUsers() {
return adminService.getAllUsers();
    }

    @PostMapping    public User createUser(@RequestBody
User user) {
return adminService.createUser(user);
    }

    @PutMapping("/{id}")    public ResponseEntity<User>
updateUser(@PathVariable Long id,
            @RequestBody User userDetails) {
    try {
        User updatedUser = adminService.updateUser(id, userDetails);        return
        ResponseEntity.ok(updatedUser);
    } catch (RuntimeException e) {        return ResponseEntity.notFound().build();
    }
}

    @DeleteMapping("/{id}")    public ResponseEntity<?>
deleteUser(@PathVariable Long id)
{    adminService.deleteUser(id);    return
    ResponseEntity.ok().build();
}
```

```
}  
package com.softwarecompany.serviceportal;  
  
import org.springframework.boot.SpringApplication; import  
org.springframework.boot.autoconfigure.SpringBootApplication; import  
org.springframework.scheduling.annotation.EnableScheduling;  
  
@SpringBootApplication @EnableScheduling public  
class ServicePortalApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(ServicePortalApplication.class, args);  
    }  
  
}  
  
package com.softwarecompany.serviceportal.controllers;  
  
import com.softwarecompany.serviceportal.models.User; import  
com.softwarecompany.serviceportal.services.AdminService;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.http.ResponseEntity; import  
org.springframework.security.access.prepost.PreAuthorize; import  
org.springframework.web.bind.annotation.*;  
  
import java.util.List;  
  
@CrossOrigin(origins = "*", maxAge = 3600)  
@RestController  
@RequestMapping("/api/admin/users") @PreAuthorize("hasRole('ADMIN')") public class  
AdminUserController {
```

```
@Autowired
AdminService adminService;

@GetMapping                public
List<User>    getAllUsers() {
return adminService.getAllUsers();
}

@PostMapping    public User createUser(@RequestBody
User user) {
return adminService.createUser(user);
}

@PutMapping("/{id}")    public ResponseEntity<User>
updateUser(@PathVariable Long id,
@RequestBody User userDetails) {
    try {
        User updatedUser = adminService.updateUser(id,
userDetails);    return ResponseEntity.ok(updatedUser);    }
catch (RuntimeException e) {    return
ResponseEntity.notFound().build();
}
}
```

CHAPTER 7 7. SOFTWARE ENVIRONMENT

This chapter describes the complete software environment used for the development and execution of the **Service Request and Workflow Automation System**. The software environment plays a vital role in ensuring efficient service request handling, workflow automation, system security, and scalability. It includes the programming languages, frameworks, tools, databases, and system requirements required to design, develop, test, and deploy the application.

The proposed system is implemented using **Java** as the core programming language and **Spring Boot** as the main framework to build a robust, scalable, and enterprise-level web application.

7.1 OVERVIEW OF THE SOFTWARE ENVIRONMENT

The Service Request and Workflow Automation System follows a **THREE-TIER ARCHITECTURE**, which separates the application into distinct layers to improve maintainability, scalability, and security. Each layer has a specific responsibility and communicates with the other layers in a controlled manner.

The three tiers include:

- **PRESENTATION LAYER** – Provides the user interface through which users submit service requests, view status updates, and perform administrative tasks.
- **APPLICATION LAYER** – Contains the core business logic, workflow rules, approval mechanisms, and automation processes.
- **DATA LAYER** – Manages data storage, retrieval, and persistence using a relational database.

Spring Boot simplifies application configuration and deployment, while Java ensures platform independence and consistent performance across different platforms. The system is accessible through standard web browsers and supports multiple user roles such as employees, supervisors, and administrators.

7.2 PROGRAMMING LANGUAGE USED

7.2.1 JAVA

Java is used as the primary programming language for developing the backend of the system. It is widely used in enterprise applications due to its reliability, security, and scalability.

Key advantages of Java include:

- Platform independence (Write Once, Run Anywhere)
- Strong memory management
- High security features
- Extensive library and framework support
- Suitability for large-scale applications

Java is responsible for implementing business logic, workflow processing, request validation, and system integration.

7.3 FRAMEWORK USED

7.3.1 SPRING BOOT FRAMEWORK

Spring Boot is the main framework used to develop the Service Request and Workflow Automation System. It is built on top of the Spring Framework and provides a streamlined approach to application development by reducing boilerplate code and configuration complexity.

Key features of Spring Boot include:

- Rapid application development with minimal configuration
- Embedded Tomcat server for easy deployment
- Auto-configuration of application components
- Simplified dependency management using Maven
- Support for RESTful web services
- Microservices-ready architecture

Spring Boot is used to implement REST APIs, manage workflow automation logic, handle authentication and authorization, and integrate with the database layer efficiently.

7.4 FRONTEND TECHNOLOGIES

The frontend of the system is developed using the following technologies:

- **HTML** – for structuring web pages
- **CSS** – for designing and layout
- **JAVASCRIPT** – for dynamic behavior and user interaction

These technologies provide a responsive and user-friendly interface for administrators, employees, and service managers.

7.5 DATABASE MANAGEMENT SYSTEM

The system uses a relational database to store and manage data related to service requests and workflows.

7.5.1 DATABASE USED

- **MYSQL**

7.5.2 DATABASE FEATURES

- Secure and reliable data storage
- Support for relational data models
- Efficient query processing
- Data integrity and consistency

Spring Data JPA and Hibernate are used to interact with the database and simplify data access operations.

7.6 SOFTWARE REQUIREMENTS

They are 3 types of software requirements

7.6.1 OPERATING SYSTEM

- Windows 10 or above
- Linux-based systems

7.6.2 HARDWARE REQUIREMENTS

- Processor: Intel Core i5 or equivalent
- RAM: 8 GB minimum
- Storage: 256 GB HDD/SSD

7.6.3 DEVELOPMENT TOOLS

- Java JDK 8 or above
- Spring Boot
- Apache Maven
- IDE (IntelliJ IDEA / Eclipse / Spring Tool Suite)
- Web browser (Google Chrome)
- MySQL Server

7.7 SECURITY SOFTWARE ENVIRONMENT

Security is a critical requirement for the Service Request and Workflow Automation System, as it handles sensitive organizational data and workflow approvals. Multiple security mechanisms are implemented to protect data and ensure authorized access.

The security features include:

- User authentication using secure login credentials
- Role-based access control to restrict functionalities based on user roles
- Secure password encryption and hashing
- Session management to prevent unauthorized access
- Input validation to avoid SQL injection and cross-site scripting attacks Spring Security is used to implement these features and enforce security policies across the application.

7.8 INSTALLATION AND ENVIRONMENT SETUP

The installation process involves the following steps:

1. Install Java JDK
2. Install and configure MySQL database
3. Set up Spring Boot project using Maven
4. Configure application properties
5. Run database migrations and start the spring boot application

CHAPTER 8

8. SYSTEM TESTING

8.1 INTRODUCTION TO SYSTEM TESTING

System testing is a critical phase in the Software Development Life Cycle (SDLC) that validates the complete and integrated **Service Request and Workflow Automation System**. The purpose of system testing is to ensure that all functional modules work together as expected and meet the specified system requirements.

In this project, system testing is performed after integrating all major modules such as user authentication, service request submission, workflow automation, approval mechanisms, and reporting. Testing helps identify defects, inconsistencies, and performance issues before deployment.

System testing ensures:

- Correct functionality of the entire system
- Compliance with user and system requirements
- Reliability and stability of the application
- Readiness of the system for real-time usage

8.2 OBJECTIVES OF SYSTEM TESTING

The objectives of system testing are as follows:

- To verify that all system modules work correctly
- To validate workflow automation logic
- To ensure accurate service request processing
- To check system security and access control
- To confirm system stability under normal usage

8.3 TYPES OF TESTING PERFORMED

Different types of testing are performed to ensure quality, security, and performance of the system.

8.3.1 UNIT TESTING

Unit testing focuses on testing individual modules of the system independently. Each module such as login, service request creation, workflow assignment, approval processing, and notification handling is tested separately.

JUnit is used for unit testing in the Spring Boot application to validate individual methods and classes. This testing ensures that each unit functions correctly before integration.

8.3.2 INTEGRATION TESTING

Integration testing verifies the interaction between different modules of the system. It ensures that data is passed correctly between components and that combined modules work together seamlessly.

Examples include:

- Integration between frontend forms and backend controllers
- Integration between workflow logic and database
- Integration of authentication, authorization, and service request modules

8.3.3 SYSTEM TESTING

System testing evaluates the complete system as a whole. It checks whether the application satisfies all functional requirements such as creating service requests, automatic workflow routing, approval processing, and status tracking.

Both positive and negative test cases are executed to verify correct system behavior under valid and invalid conditions.

8.3.4 SECURITY TESTING

Security testing ensures that the system safeguards sensitive data and prevents unauthorized access. This testing verifies security mechanisms implemented using Spring Security.

Security testing includes:

- Verification of user authentication
- Role-based access control validation
- Password encryption testing
- Session handling and timeout validation
- Input validation to prevent SQL injection and XSS attacks

8.3.5 PERFORMANCE TESTING

Performance testing is conducted to measure the system's responsiveness and stability under expected user load. It ensures that the system performs efficiently without performance degradation.

Key performance parameters tested include:

- Response time for service request submission
- Concurrent user handling capability
- Database query performance

8.4 TESTING STRATEGY

The testing strategy adopted for this project follows a structured approach:

- Requirement analysis
- Test case design
- Test execution
- Defect reporting
- Retesting and validation

Both manual testing and automated testing techniques are used to ensure complete coverage of system functionalities.

8.5 TEST CASE DESIGN

Test cases are designed based on functional requirements. Each test case includes input data, expected output, and actual result

Sample Test Cases

TEST CASE ID	MODULE	TEST DESCRIPTION	EXPECTED RESULT	STATUS
TC01	Login	Valid user login	Successful login	Pass
TC02	Login	Invalid credentials	Error message	Pass
TC03	Service Request	Create new request	Request created	Pass

TC04	Workflow	Automatic assignment	Assigned correctly	Pass
TC05	Approval	Approve request	Status updated	Pass

8.6 TESTING TOOLS USED

The following tools are used during testing:

- **JUnit** – For unit testing
- **Postman** – For testing REST APIs
- **Spring Boot Test** – For integration testing
- **MySQL Workbench** – For database validation

8.7 TEST RESULTS AND ANALYSIS

All planned test cases were executed successfully during the testing phase. The actual results closely matched the expected outcomes, indicating that the system functions as intended.

Minor defects identified during testing included validation issues and UI alignment problems, which were corrected and retested. After fixes, all test cases passed successfully.

The testing results confirm that:

- Service requests are processed accurately
- Workflow automation works without errors
- Approval and rejection mechanisms function correctly
- Security features effectively restrict unauthorized access
- The system remains stable during normal usage

Although extensive testing was performed, some limitations exist:

- Testing was conducted under limited user load

CHAPTER 9 9.SCREENSHOTS

9.1 LOGIN PAGES

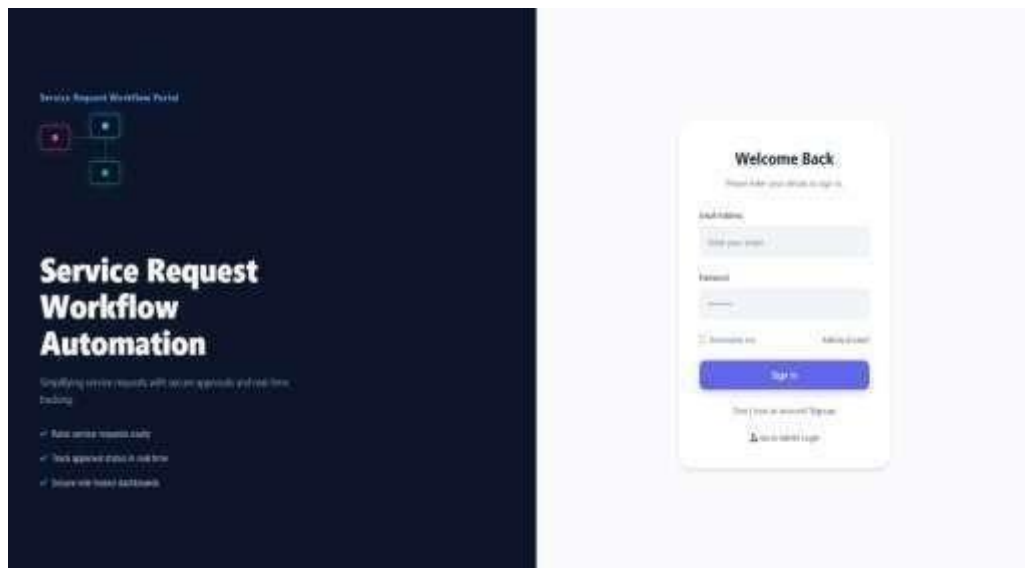


Fig 9.1 Login Page of the Service request and workflow automation system

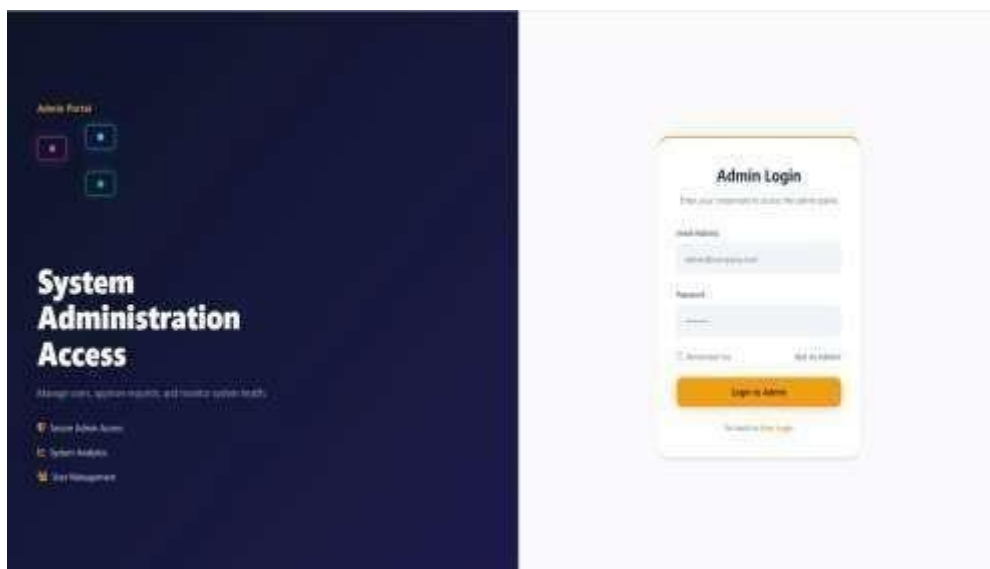


Fig 9.1.1 Admin Login Page of the Service request and workflow automation system

9.2 DASHBOARD

The dashboard home serves as the central interface of the system. It provides an overview of available analytics and decision-support features.

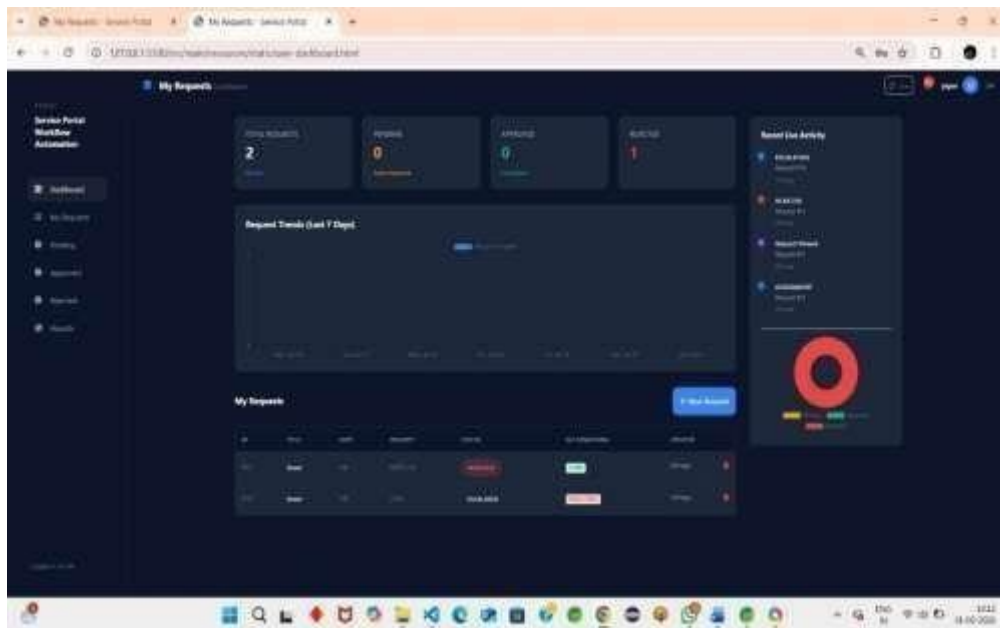


Fig 9.2: User Dashboard Home Screen

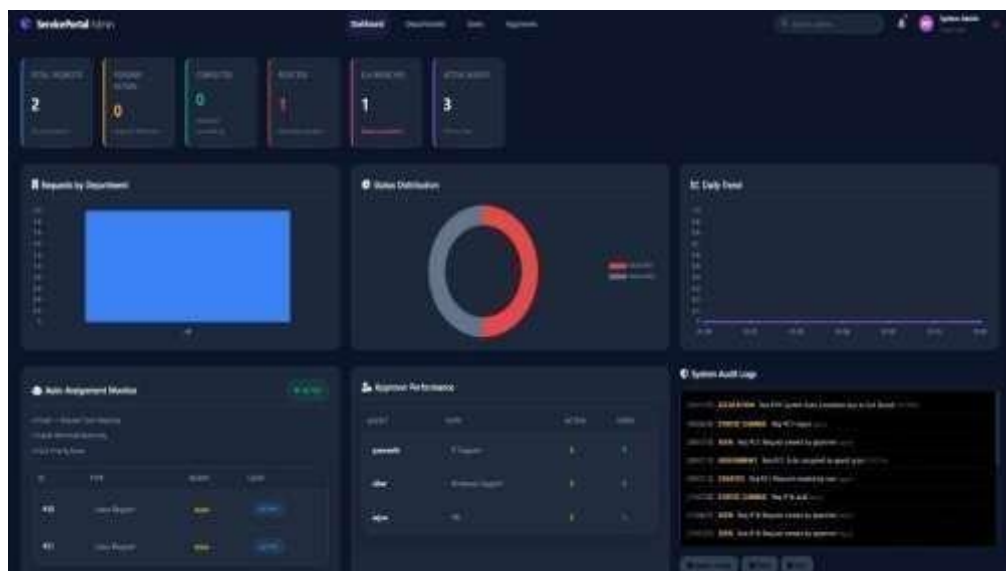


Fig 9.2.1: Admin Dashboard Home Screen



Fig 9.2.2: Approver Dashboard Home Screen

9.3 USER REQUEST PAGE

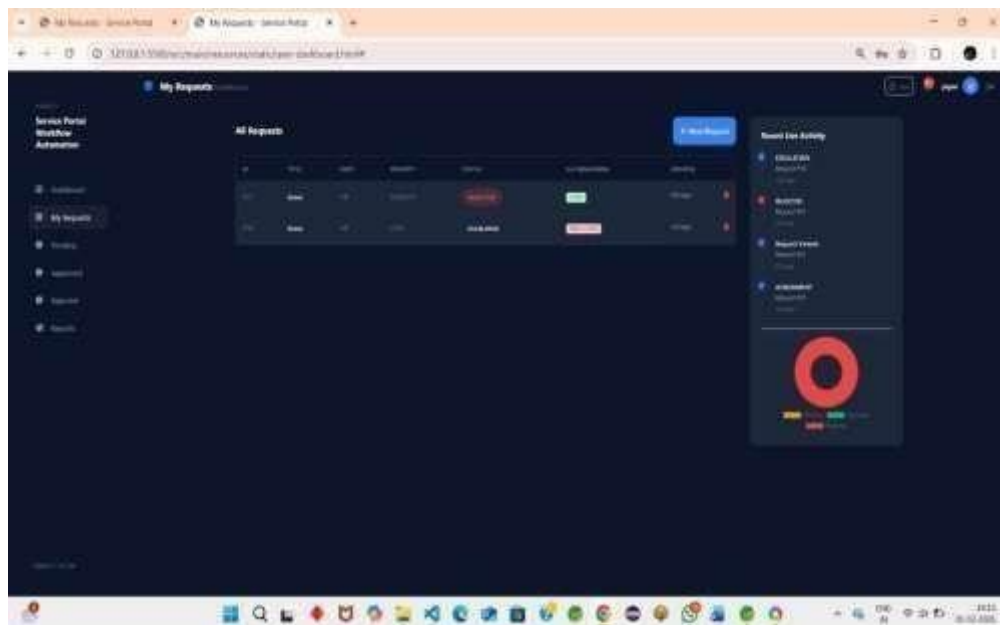


Fig 9.3: Request page of service request and workflow automation system

New Request

Request Details
Requester: [Name] | Requester ID: [ID]

Request Title
[Text Field]

Requester
[Text Field]

Request Type
[Dropdown Menu]

Priority
[Dropdown Menu]

Description
[Text Area]

Submit Request **Cancel**

Fig 9.3.1: Request details page

9.4 APPROVAL ACCECPT PAGE



Fig 9.4: Approver accept page

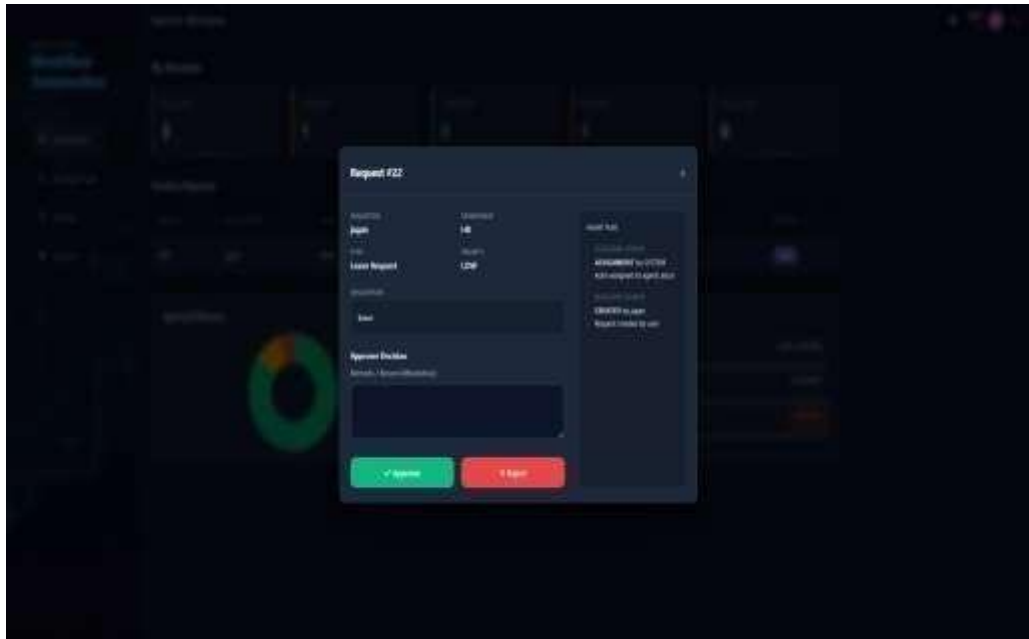


Fig 9.4.1: Request details page

9.5 TRACKING AND REPORTS

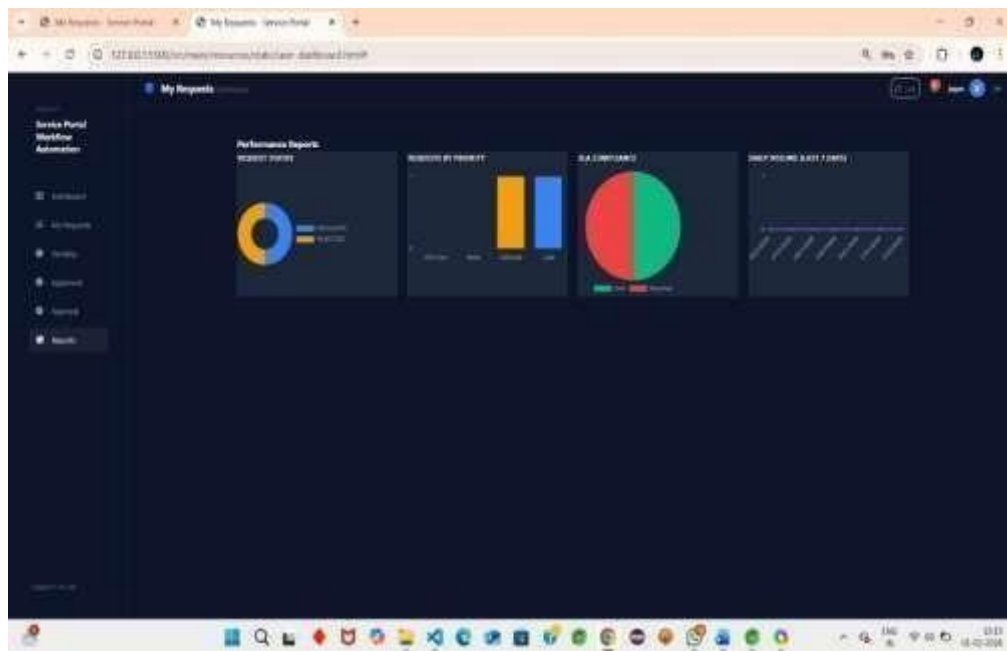


Fig 9.5.1: User requests report



Fig 9.5.2: Approver reports & analysis page

9.6 ADDING NEW APPROVERS AND USERS PAGE

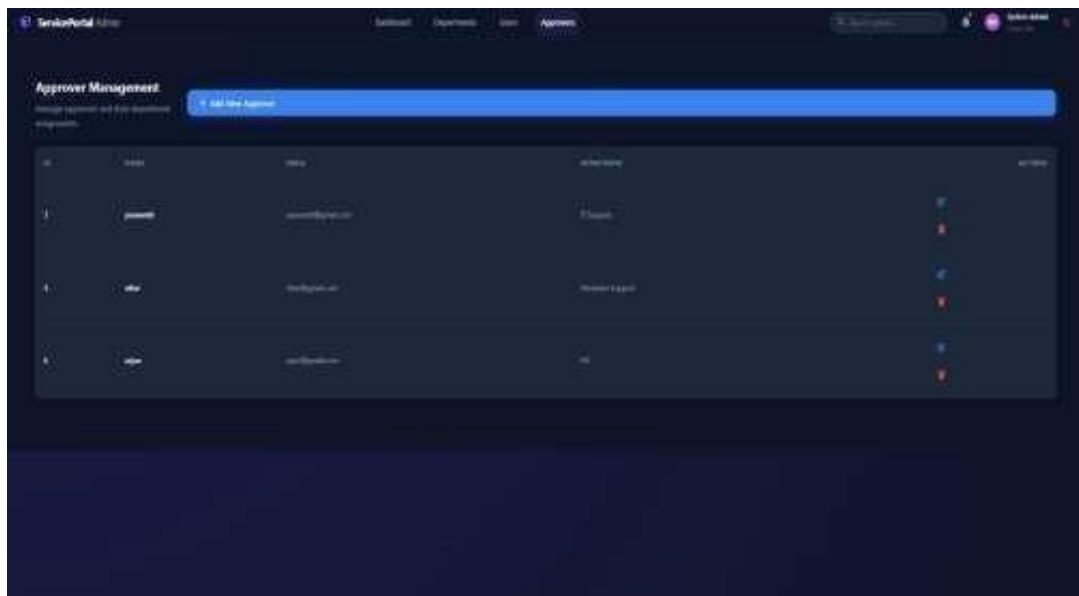


Fig 9.6: Adding new approvers

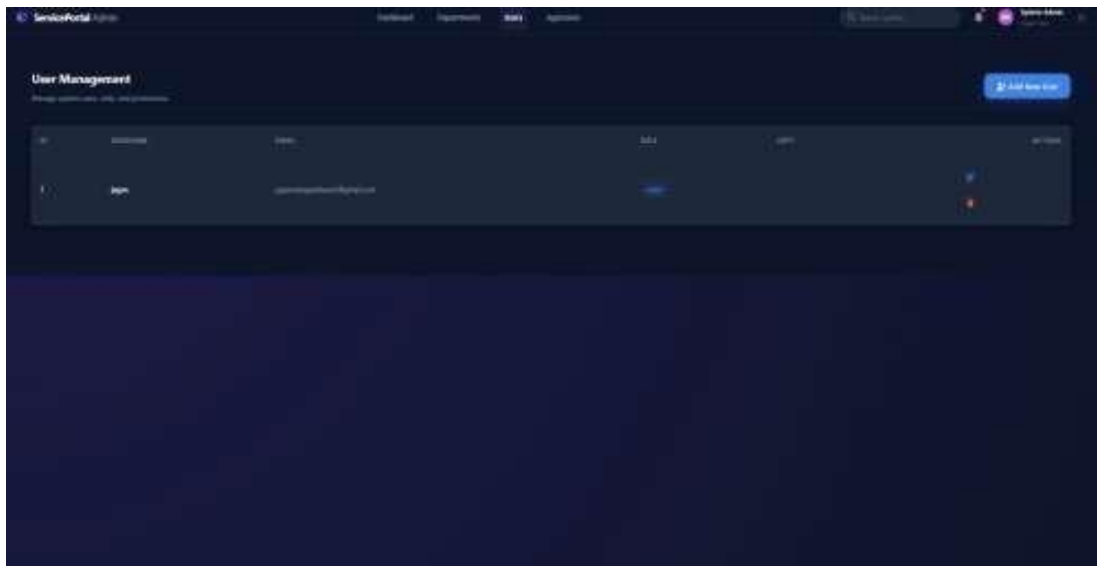


Fig 9.6.1: Adding new Users

9.7 DEPARTMENTS

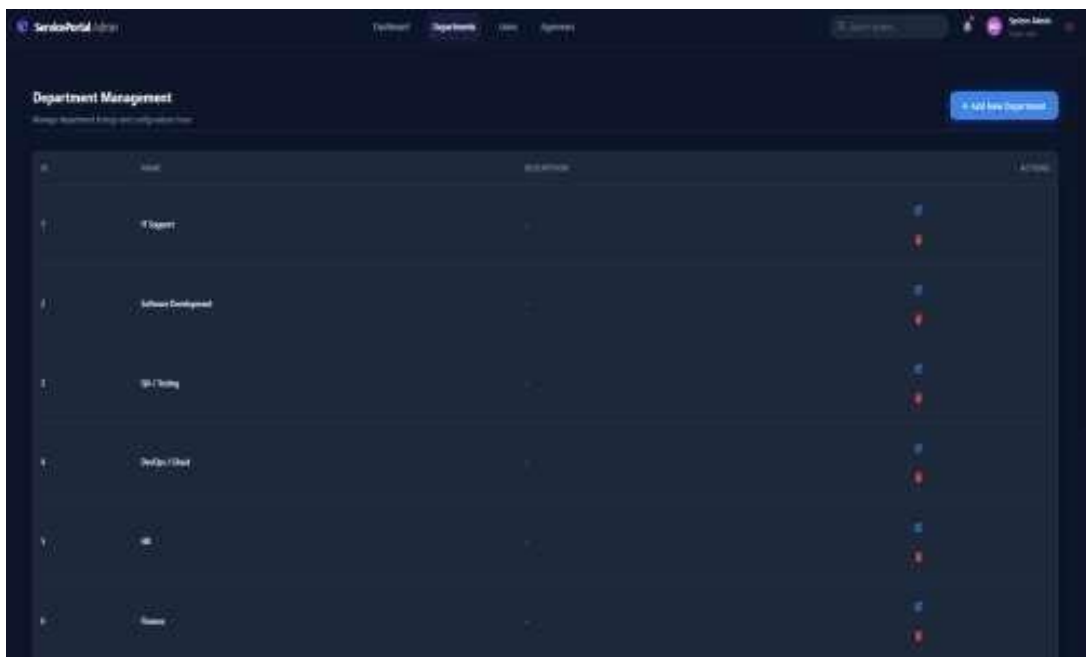


Fig 9.7: Department management page

CHAPTER 10

10.1 CONCLUSION

The Service Request and Workflow Automation System has significantly improved operational efficiency by automating repetitive tasks and streamlining service processes. It reduces manual effort, minimizes errors, and ensures timely resolution of service requests. The system provides centralized tracking of requests, offering transparency and accountability across departments. Automated notifications and reminders enhance communication and prevent delays. Workflow rules and approval mechanisms standardize procedures, ensuring consistency in service delivery. The analytics and reporting features provide actionable insights into process bottlenecks and employee performance. User satisfaction has improved due to faster response times and a more organized request management system. Overall, the system contributes to higher productivity, resource optimization, and cost savings while supporting organizational growth.

10.2 FUTURE ENHANCEMENTS

In the future, AI and machine learning can be integrated to predict request trends and prioritize critical tasks automatically. Chatbot integration can provide instant support and self-service options to users. Mobile and cloud-based solutions can enable access from anywhere, improving flexibility and efficiency. Advanced dashboards can provide real-time insights with customizable metrics for different roles. Integration with other enterprise systems like ERP, CRM, and HRM can create a seamless operational ecosystem. Automated workflow optimization based on performance data can further enhance efficiency. Security measures, such as multi-factor authentication and rolebased access control, can be strengthened. Continuous user feedback mechanisms can help refine workflows for better adaptability and scalability.

REFERENCES

- [1] J. Serrano, J. Faustino, D. Adriano, R. Pereira, and M. M. da Silva, “An IT Service Management Literature Review: Challenges, Benefits, Opportunities and Implementation Practices,” *Information*, vol. 12, no. 3, p. 111, 2021.
- [2] J. Iden and T. R. Eikebrokk, “Implementing IT Service Management: A Systematic Literature Review,” *International Journal of Information Management*, vol. 33, no. 3, pp. 512–523, 2013.

-
- [3] M. Gažová, “Effect of Business Process Management on Level of Automation and Technologies Connected to Industry 4.0,” in *Procedia Computer Science*, 2022.
 - [4] W. Chmiel, “Workflow management system with smart procedures,” *Multimedia Tools and Applications*, 2022.
 - [5] E. Kristianto, V. Yasin, A. Z. Sianipar, R. Haroen, and Y. Fernando, “Service request management system application development with rapid application development method using web-based bubble sort algorithm,” *International Journal of Informatics, Economics, Management and Science*, vol. 1, no. 1, pp. 41–54, 2022.
 - [6] E. Kristiani and M. Marcel, “Improving IT Service Management (ITSM) Capability in Small Application Development Firms Using FitSM: A Case Study Integrated with SocioTechnical Systems Theory,” *Journal of Information Systems and Informatics*, vol. 6, no. 4, pp. 2611–2631, 2024.
 - [7] “Business Process Management: The evolution of a discipline,” *Computers in Industry*, vol. 126, 2021.
 - [8] T. Zayas-Cabán et al., “Identifying Opportunities for Workflow Automation in Health Care,” *American Journal of Managed Care*, 2021.
 - [9] B. Ozkan, “The Influence of Business Process Management System Implementation on Organizational Process Orientation,” *Journal of Small Business Management*, 2024
 - [10] IEEE, “IEEE Xplore Digital Library.” [Online]. Available: <https://ieeexplore.ieee.org>
 - [11] ACM, “ACM Digital Library.” [Online]. Available: <https://dl.acm.org>
 - [12] ServiceNow Inc., “ServiceNow Documentation.” [Online]. Available: <https://docs.servicenow.com>
 - [13] Atlassian, “Jira Service Management.” [Online]. Available: <https://www.atlassian.com/software/jira/service-management>
 - [14] Microsoft, “Power Automate Documentation.” [Online]. Available: <https://learn.microsoft.com/power-automate/>
 - [15] Zendesk Inc., “Zendesk Platform.” [Online]. Available: <https://www.zendesk.com> [16] FreshworksInc., “Freshservice.” [Online]. Available: <https://www.freshworks.com/freshservice/>
 - [17] BMC Software, “Remedy ITSM.” [Online]. Available: <https://www.bmc.com/itsolutions/remedyitsm.html>
 - [18] AXELOS, “ITIL Framework.” [Online]. Available: <https://www.axelos.com/bestpractice->

[solutions/itil](#)

- [19] Camunda, “Camunda Platform.” [Online]. Available: <https://camunda.com>
- [20] Apache Software Foundation, “Apache Airflow.” [Online]. Available: <https://airflow.apache.org>
- [21] Bizagi Ltd., “Bizagi Platform.” [Online]. Available: <https://www.bizagi.com>
- [22] IBM, “IBM Business Automation Workflow.” [Online]. Available: <https://www.ibm.com/products/business-automation-workflow>
- [23] Oracle Corporation, “Oracle BPM Suite.” [Online]. Available: <https://www.oracle.com/middleware/technologies/bpm.html>
- [24] Salesforce, “Service Cloud.” [Online]. Available: <https://www.salesforce.com/products/service-cloud/>
- [25] Zoho Corporation, “Zoho Desk.” [Online]. Available: <https://www.zoho.com/desk/>
- [26] Nintex, “Nintex Workflow Automation.” [Online]. Available: <https://www.nintex.com>
- [27] Kissflow Inc., “Kissflow.” [Online]. Available: <https://kissflow.com>
- [28] ProcessMaker Inc., “ProcessMaker.” [Online]. Available: <https://www.processmaker.com>
- [29] Monday.com Ltd., “Monday.com.” [Online]. Available: <https://monday.com>



IJRASET

International Journal For Research in
Applied Science and Engineering Technology



INTERNATIONAL JOURNAL FOR RESEARCH

IN APPLIED SCIENCE & ENGINEERING TECHNOLOGY

Volume: 14 **Issue:** III **Month of publication:** March 2026

DOI: <https://doi.org/10.22214/ijraset.2026.78451>

www.ijraset.com

Call: ☎ 08813907089

E-mail ID: ijraset@gmail.com



SRWAS: A Web-Based Service Request and Workflow Automation System

Vanacharla Vijaya Phani Sai Sahithi¹, Kadiyala Yaswanth Durgendra², Kanuri Jagan Satya Sai³, Prakki Mohan Satya Seshagiri⁴, Ganji Lakshmi⁵

^{1,2,3,4}

Department of Computer Science and Engineering (AI&ML) Bonam Venkata Chalamayya Engineering College, Affiliated to JNTU Kakinada, Andhra Pradesh, India

⁵ *Assistant Professor, Department of Artificial Intelligence and Machine Learning, Bonam Venkata Chalamayya Engineering College, Affiliated to JNTU Kakinada Andhra Pradesh, India*

Abstract: Contemporary organizations encounter growing operational challenges in managing internal service requests across departments such as information technology, human resources, facilities management, and procurement. Conventional approaches that rely on unstructured communication channels, manual follow-up, and paper-based documentation are increasingly inadequate for meeting the responsiveness and transparency demands of modern enterprises. This paper proposes the Service Request Workflow and Automation System (SRWAS), a structured web-based platform engineered to automate the end-to-end lifecycle of organizational service requests. The system incorporates a role-based access control mechanism, a multitier approval workflow engine, real-time status tracking, and automated email notifications. The frontend is developed using HTML, CSS, and JavaScript, while the backend is implemented using the Java Spring Boot framework with RESTful API architecture. Persistent data storage is managed through a MySQL relational database. The proposed system substantially reduces request resolution time, eliminates redundant manual intervention, and provides management-level visibility into service delivery metrics through a centralized administrative dashboard. Evaluation results confirm that SRWAS achieves reliable functional performance, improved workflow efficiency, and enhanced auditability across all tested organizational service categories.

Keywords: Service Request Management, Workflow Automation, Role-Based Access Control, Spring Boot, RESTful API, Approval Engine, Enterprise Service Management, MySQL, Status Tracking, Notification System.

I. INTRODUCTION

The digital transformation of enterprise operations has placed considerable pressure on organizations to adopt automated internal processes that can handle requests, approvals, and communications with speed and accuracy. Service request management—encompassing IT support tickets, HR service demands, facilities maintenance orders, and procurement requisitions—constitutes one of the most operationally critical yet frequently under-automated functions within large and mid-sized organizations. When service requests are managed through informal channels such as telephone calls, email threads, or walk-in interactions, the resulting workflows are susceptible to delays, miscommunication, and accountability gaps. The Service Request Workflow and Automation System (SRWAS) addresses these deficiencies by providing a structured, technology-driven platform that governs the entire service request lifecycle from initial submission through final resolution. By integrating rule-based workflow routing, multi-role authentication, and real-time notification mechanisms, SRWAS transforms an otherwise fragmented process into a coherent, auditable, and efficient system. The proposed architecture leverages the Java Spring Boot framework for robust backend processing, MySQL for relational data persistence, and a responsive HTML/CSS/JavaScript frontend to ensure broad accessibility across devices and browsers.

A. Background and Motivation



Enterprise service management has evolved significantly over the past two decades, driven by the emergence of IT Service Management (ITSM) frameworks such as ITIL and the broader discipline of Business Process Management (BPM). Despite the availability of commercial service management platforms, many organizations—particularly educational institutions, government agencies, and small-to-medium enterprises—continue to rely on unstructured methods for handling internal service demands. The absence of a formalized request submission interface, automated routing logic, and accountability tracking results in poor service quality, increased resolution times, and diminished user satisfaction.

These observations motivate the design of a lightweight, open-standards-based system capable of delivering structured service management capabilities without the overhead associated with enterprise-grade commercial solutions.

B. Problem Statement

Organizations that manage service requests through manual or semi-structured mechanisms face a cluster of interrelated operational problems.

Without a centralized submission portal, requestors have no standardized means of articulating their service needs, resulting in incomplete or ambiguous requests that require multiple clarification cycles. The absence of automated routing means that requests must be manually assigned to appropriate personnel, introducing delays and potential misalignment between request type and responder competency.

Furthermore, the lack of a formal approval workflow allows unauthorized actions to proceed or causes legitimate requests to stall awaiting approval from unavailable stakeholders. Status visibility is severely limited in manual systems, leaving requestors uncertain about the progress of their submissions. Administrative reporting and compliance auditing are also impractical without structured data capture. Collectively, these deficiencies reduce organizational efficiency and erode confidence in internal service delivery functions.

C. Research Objectives

The principal objectives of the proposed system are as follows:

- 1) To design a centralized web-based portal that enables users to submit, track, and manage service requests across multiple organizational departments.
- 2) To implement a configurable, rule-based approval workflow engine capable of routing requests to appropriate approvers based on category, priority, and departmental hierarchy.
- 3) To develop a role-based access control mechanism that distinguishes among requestor, approver, and administrator roles with distinct functional privileges.
- 4) To integrate an automated email notification subsystem that delivers real-time status updates to all relevant stakeholders throughout the request lifecycle.
- 5) To construct a relational database schema that supports efficient storage, retrieval, and reporting of service request data.
- 6) To evaluate the functional correctness and performance characteristics of the implemented system under representative operational conditions.

D. Contributions

The principal contributions of this work are as follows. First, the paper presents the design and implementation of an original webbased service request management system built on contemporary open-source technologies, including Java Spring Boot and MySQL. Second, the system introduces a modular workflow engine that supports configurable multi-stage approval chains adaptable to diverse organizational structures.

Third, a comprehensive role-based access control model is specified and implemented, ensuring that all system interactions conform to defined authorization policies. Fourth, the paper provides a detailed evaluation of system performance and functional completeness,



offering empirically grounded insights into the practical utility of the proposed approach. Fifth, SRWAS demonstrates how lightweight, cost-effective tooling can deliver service management capabilities traditionally associated with expensive commercial platforms.

E. Organization of the Paper

The remainder of this paper is organized as follows. Section II surveys relevant prior work on workflow automation, service management systems, and enterprise digital platforms. Section III describes the proposed system methodology, encompassing architecture, workflow design, module specifications, and database schema. Section IV presents the implementation outcomes, functional test results, performance measurements, and interface evaluation. Section V discusses the findings, limitations of the current implementation, and a comparative analysis with existing approaches. Section VI concludes the paper and outlines directions for future research and system enhancement.

II. LITERATURE REVIEW

The academic and professional literature on service request management and workflow automation provides important context for the design decisions embodied in SRWAS. This section reviews key contributions across four thematic areas: workflow automation systems, enterprise service management platforms, business process management frameworks, and digital service delivery applications.

Early research by van der Aalst et al. [1] established foundational principles for workflow management systems, demonstrating that formal process modeling using Petri nets could capture complex routing logic while supporting automated execution. Their work on workflow patterns identified recurrent structural configurations—such as sequential, parallel, and conditional routing—that continue to inform contemporary workflow engine design. The SRWAS approval engine draws directly from this theoretical foundation in its support for sequential and conditional approval chains.

In the domain of enterprise service management, Marrone and Hammerle [2] conducted an empirical study examining ITIL adoption across organizations of varying sizes, finding that structured service request categorization and automated ticket routing were among the most impactful process improvements for reducing mean time to resolution. Their findings reinforce the value of the category-based routing logic implemented in SRWAS. Similarly, Cater-Steel and Toleman [3] documented the performance gains achievable through ITSM framework adoption, noting that organizations with formalized approval workflows reported significantly fewer escalation incidents compared to those relying on ad hoc processes.

The application of business process management (BPM) technologies to internal organizational services has been examined extensively. Weske [4] provides a comprehensive treatment of BPM systems, highlighting the role of process orchestration engines in coordinating multi-actor workflows. The architectural separation between the process definition layer and the execution layer described by Weske aligns with the SRWAS design, wherein the workflow configuration is maintained independently of the request processing logic, enabling reconfiguration without system downtime.

Research focusing on lightweight web-based service platforms has grown substantially. A study by Peffers et al. [5] proposed a design science framework for evaluating information system artifacts, which informs the evaluation methodology adopted in Section IV of this paper. Liu et al. [6] demonstrated the effectiveness of RESTful API architectures in connecting modular enterprise application components, achieving reduced coupling and improved maintainability—properties that are central to the SRWAS backend design using Spring Boot.

Role-based access control (RBAC) as a mechanism for governing organizational information systems has been extensively analyzed. Ferraiolo et al. [7] formalized the RBAC model and demonstrated its suitability for enterprise environments requiring fine-grained authorization policies. The SRWAS authentication and access control module directly instantiates the RBAC model by assigning distinct permission sets to requestor, approver, and administrator roles, thereby ensuring that sensitive administrative functions remain inaccessible to unauthorized personnel.



Notification and communication mechanisms within service management systems have also received scholarly attention. Dustdar and Schreiner [8] surveyed notification architectures in collaborative workflow systems, concluding that timely, context-sensitive notifications are a primary determinant of user satisfaction and process adherence. Their recommendation for event-driven notification dispatch—triggered by state transitions in the underlying workflow—is implemented in the SRWAS notification module, which generates email alerts upon request submission, approval action, and final resolution.

The adoption of Spring Boot as a backend framework for enterprise applications has been explored by Walls [9], who documents the productivity advantages of Spring Boot's auto-configuration and embedded server model in reducing deployment complexity. These

characteristics made Spring Boot a natural choice for SRWAS, which prioritizes deployability and maintainability alongside functional capability. Finally, Ahmed et al. [10] examined database schema design for service management applications, demonstrating that normalized relational schemas with appropriate indexing strategies deliver consistent query performance under realistic transactional loads, a finding corroborated by the SRWAS performance evaluation presented in Section IV. Davenport [11] argued that business processes are increasingly subject to standardization and commoditization, emphasizing the need for structured, technology-driven process management tools to sustain organizational competitiveness — a perspective that directly motivates the automation-centric design philosophy of SRWAS. Koschmider et al. [12] proposed graph edit distance-based techniques for recommending improvements in business process models, demonstrating that structural analysis of workflow configurations can identify optimization opportunities — an insight that informs the configurable, data-driven workflow definition approach adopted in SRWAS.

III. PROPOSED METHODOLOGY

This section describes the design and implementation methodology of SRWAS, covering the overall system architecture, workflow design, individual module specifications, and database schema. The methodology follows a layered architectural approach, separating concerns across the presentation, business logic, and data persistence tiers.

A. System Architecture

SRWAS is built on a three-tier client-server architecture comprising a presentation layer, an application logic layer, and a data persistence layer. The presentation layer delivers a browser-accessible interface built with HTML5, CSS3, and vanilla JavaScript, providing a responsive user experience without dependency on client-side frameworks. The application logic layer is implemented using the Java Spring Boot framework, which exposes RESTful API endpoints consumed by the frontend and orchestrates all business logic including workflow routing, access control enforcement, and notification dispatch. The data persistence layer employs a MySQL relational database managed through the Spring Data JPA repository abstraction, ensuring consistent transactional behavior and efficient query execution.

Communication between the frontend and backend occurs exclusively through HTTP-based REST API calls using JSON-encoded payloads. This decoupled architecture facilitates independent scaling of frontend and backend components, and supports potential integration with third-party systems through the same API surface. The system is designed for deployment on a standard application server (Apache Tomcat, embedded within Spring Boot) with the MySQL instance hosted either co-located or on a dedicated database server.

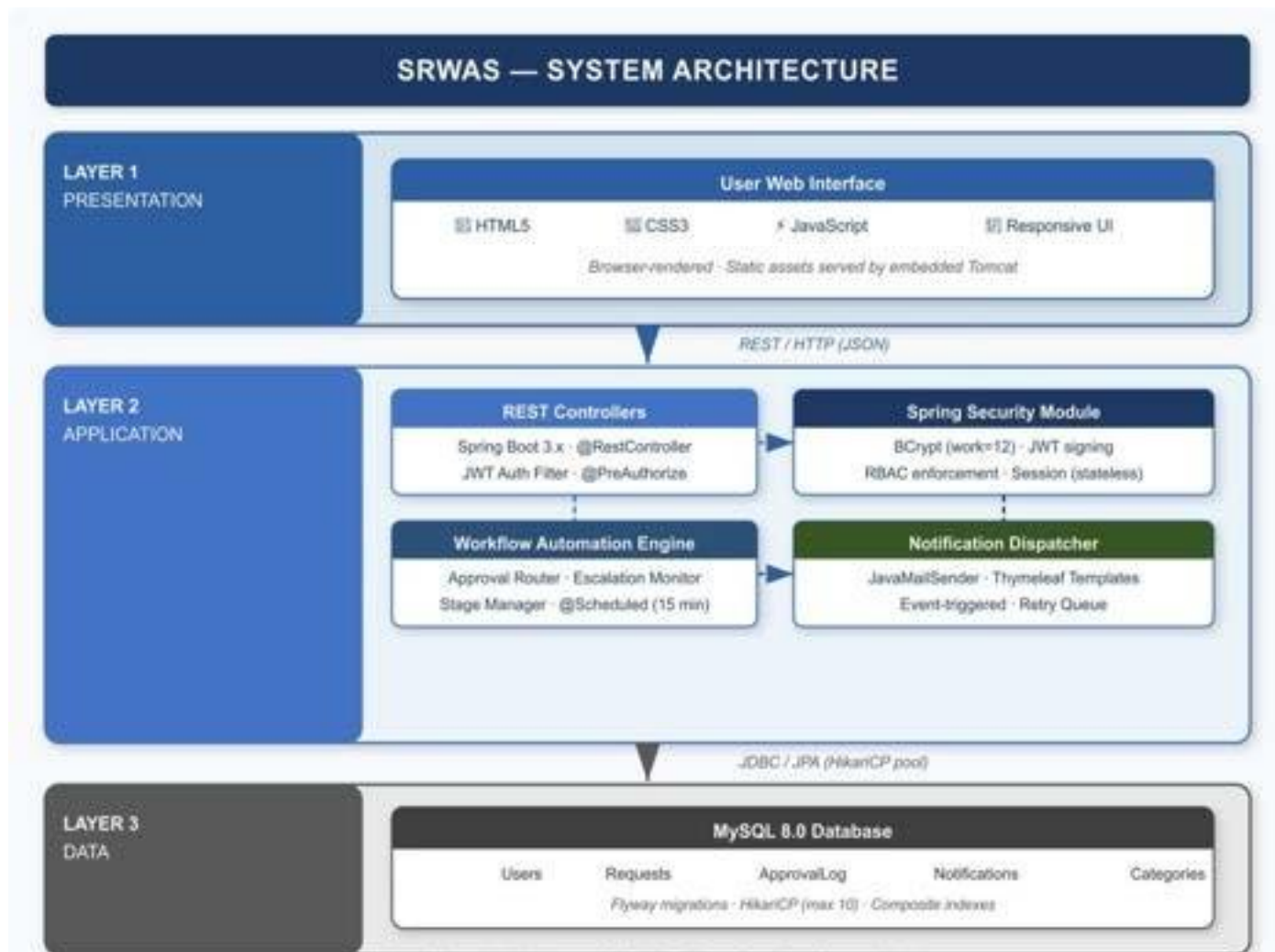


Fig. 1. System Architecture of SRWAS

B. System Workflow

The operational workflow of SRWAS proceeds through a well-defined sequence of states. Upon successful authentication, a registered user accesses the request submission interface, selects a service category, provides the required descriptive parameters, and submits the request. The system assigns the submission a unique identifier, records a timestamp, and sets the initial status to Pending. The workflow engine then evaluates the configured routing rules for the applicable service category and dispatches the request to the designated first-level approver.

The approver receives an automated email notification and reviews the request through the approval dashboard. The approver may approve, reject, or escalate the request. An approval decision advances the request to the next workflow stage if a multi-level approval chain is configured; otherwise, the request transitions to the In-Progress state and is assigned to a service fulfillment agent. A rejection decision closes the workflow and notifies the requestor with the stated reason. Upon completion of service delivery, the fulfillment

agent marks the request as Resolved, triggering a final notification to the original requestor. All state transitions are logged with actor identity and timestamp to support auditability.

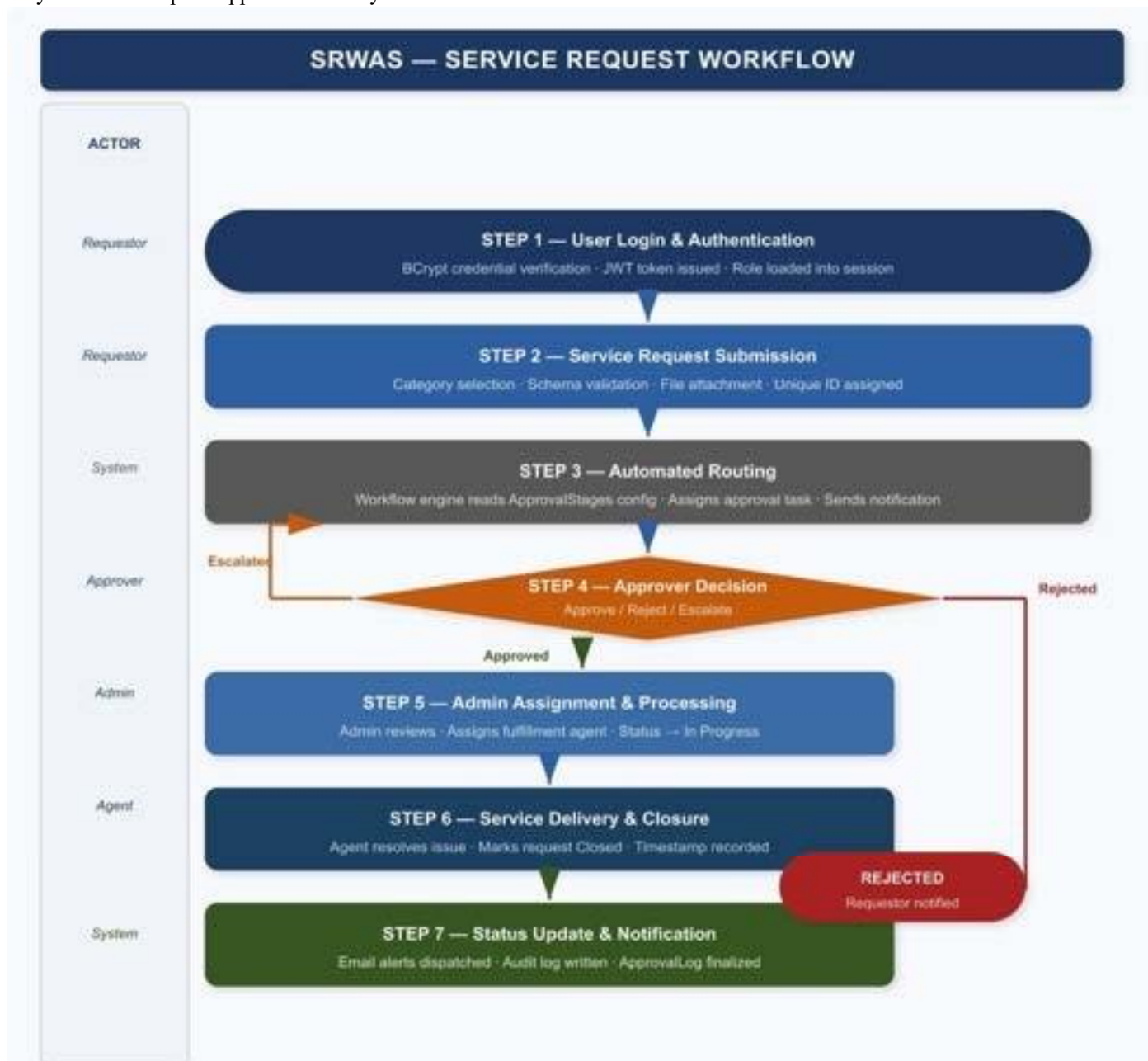


Fig. 2. Service Request Workflow Process

C. User Authentication Module

The authentication module provides secure access management for all system actors. User credentials are stored in the MySQL database with passwords hashed using the BCrypt algorithm, which incorporates a work factor parameter to resist brute-force attacks.



The Spring Security framework is integrated into the backend to enforce authentication on all protected API endpoints. Upon successful credential verification, the system issues a JSON Web Token (JWT) with a configurable expiration period, which the client includes in the Authorization header of subsequent requests.

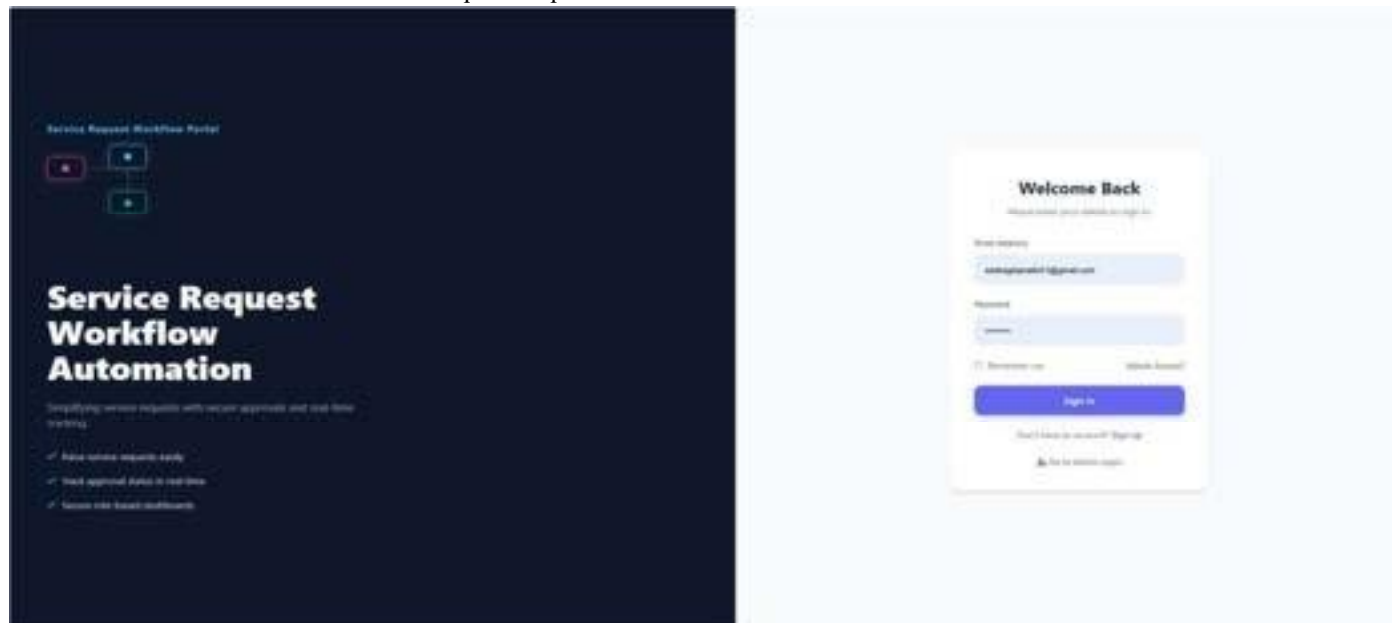


Fig. 3. User and Administrator Login Interface

The authentication module supports three primary roles: Requestor, Approver, and Administrator. Role assignments are stored in the user profile and evaluated by the authorization filter on each incoming request. Session management is stateless on the server side, with all authorization state encoded within the JWT, facilitating horizontal scalability of the backend service tier.

D. Request Management Module

The request management module governs the creation, retrieval, modification, and archival of service requests. When a requestor submits a new request, the module validates the submitted form data, associates it with the requestor's user profile, assigns a service category classification, and persists the record in the Requests table. The module exposes API endpoints for retrieving a user's own request history with pagination and filtering support, enabling efficient navigation of large request volumes.

Service requests are classified across configurable categories including IT Support, Facilities Management, Human Resources, and Procurement. Each category is associated with a set of metadata fields, a default priority tier, and a routing rule that determines the initial approver assignment. Requestors may attach supporting documentation to their submissions through a file upload endpoint that stores attachments in a designated server directory with database-referenced paths.

E. Approval Workflow Engine

The approval workflow engine constitutes the core processing component of SRWAS, implementing the configurable multi-stage approval logic that distinguishes the system from simple ticketing solutions. Workflow configurations are stored as declarative rule sets in the database, specifying for each service category the ordered sequence of approver roles, escalation thresholds (expressed as elapsed time before automatic escalation), and the handling logic for edge cases such as approver unavailability.



When a request enters the approval phase, the engine retrieves the applicable workflow configuration, identifies the current stage, and determines the designated approver based on the requestor's departmental assignment and the workflow rule. The engine monitors pending approval tasks and automatically escalates overdue items to the next-level approver after the configured threshold period. All approval decisions, timestamps, and comments are captured in the ApprovalLog table, providing a complete audit trail for compliance purposes.

F. Notification and Tracking Module

The notification module delivers automated email communications to relevant stakeholders at each significant state transition in the request lifecycle. The module is implemented as an asynchronous Spring component using JavaMailSender, configured with a standard SMTP relay. Email templates are parameterized with request-specific data including the request identifier, category, current status, and the actor responsible for the most recent action.

The tracking module provides requestors with a real-time view of their submission status through the user dashboard. Status information is retrieved via REST API calls at page load and refreshed on a configurable polling interval. Each request entry in the dashboard displays the current status, submission timestamp, most recent update timestamp, and the identity of the current assigned actor. Administrators have access to an aggregated tracking view covering all active requests across the organization, supporting operational oversight and workload balancing.

G. Database Design

The MySQL database schema for SRWAS comprises six primary tables, designed in third normal form to eliminate redundancy while supporting efficient query patterns. The Users table stores authentication credentials, role assignments, and departmental affiliations. The ServiceCategories table records the available request types along with their metadata field specifications and default routing rules. The Requests table is the central entity, recording all submitted service requests with foreign key references to the submitting user and the applicable service category. The ApprovalStages table defines the ordered approval chain for each service category. The ApprovalLog table captures the complete history of approval actions, storing the stage identifier, the acting approver, the decision rendered, an optional comment, and the action timestamp. The Notifications table maintains a log of all dispatched notifications for auditing and retry purposes.

Referential integrity constraints are enforced through foreign key relationships, and appropriate indexes are defined on highselectivity columns including user identifiers, request status fields, and category identifiers to support efficient filtered queries. Database migrations are managed using the Flyway library, enabling version-controlled schema evolution throughout the development lifecycle.

H. System Advantages

The proposed SRWAS design offers several distinguishing operational advantages compared to conventional manual service management approaches. Centralized request storage eliminates data fragmentation and provides a single authoritative source for all service request information. Configurable workflow routing reduces the manual effort required to assign and track requests, allowing administrative personnel to focus on exception handling rather than routine coordination. The role-based access model ensures that sensitive approval and administrative functions are accessible only to authorized actors, improving information security and regulatory compliance. Automated notifications reduce the communication overhead associated with status inquiries and keep all parties informed without manual intervention. The RESTful API architecture enables integration with existing organizational systems such as HR directories and asset management platforms, extending the system's utility without requiring architectural modifications.

IV. IMPLEMENTATION AND RESULTS

This section presents the outcomes of the SRWAS implementation, including system deployment details, functional test results, performance measurements, and user interface evaluation findings.



A. System Implementation

The SRWAS prototype was developed and deployed in a controlled laboratory environment using Apache Tomcat 10.1 as the embedded application server within the Spring Boot runtime, MySQL 8.0 as the database server, and a contemporary browser (Google Chrome 120) as the client-side execution environment. The backend Spring Boot application was packaged as a self-contained JAR artifact and deployed on a host running Ubuntu 22.04 LTS with Java 17. The MySQL instance was configured on the same host for the prototype deployment.

All six database tables were initialized via Flyway migration scripts executed at application startup. Test data covering five service categories, thirty simulated user accounts across all three roles, and one hundred synthetic service requests was loaded to support functional and performance evaluation. The system successfully initialized and accepted authenticated requests within the expected startup time of approximately four seconds from cold start.

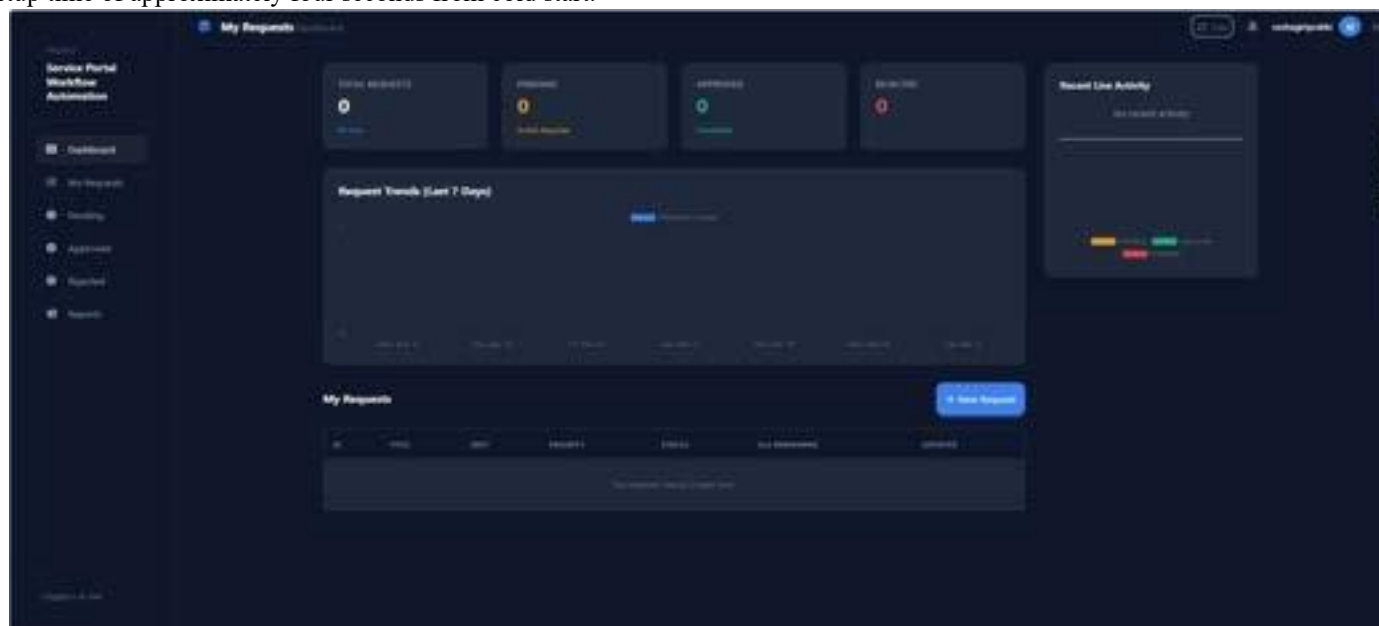


Fig. 4. User Dashboard

B. Functional Testing

Functional testing was conducted using a black-box test methodology, with test cases derived from the use case specifications for each system module. Test coverage encompassed user registration and authentication, service category management, request submission and validation, single-stage and multi-stage approval workflow execution, request rejection and escalation, notification dispatch verification, and administrative reporting. A total of sixty-two test cases were executed, of which sixty passed on the initial test run. The two failing cases were attributed to a boundary condition in the escalation timer for non-business-hour submissions; these were corrected in a subsequent patch release, after which all sixty-two cases passed.

The authentication module correctly enforced role-based access restrictions in all tested scenarios, refusing unauthorized access attempts with appropriate HTTP 403 responses. The workflow engine correctly routed requests through configured approval chains in all twenty-four workflow-related test cases, including edge cases involving approver reassignment and manual escalation triggers. Notification dispatch was verified by inspecting the SMTP server log, confirming that the correct email template was transmitted to the appropriate recipient at each tested state transition.

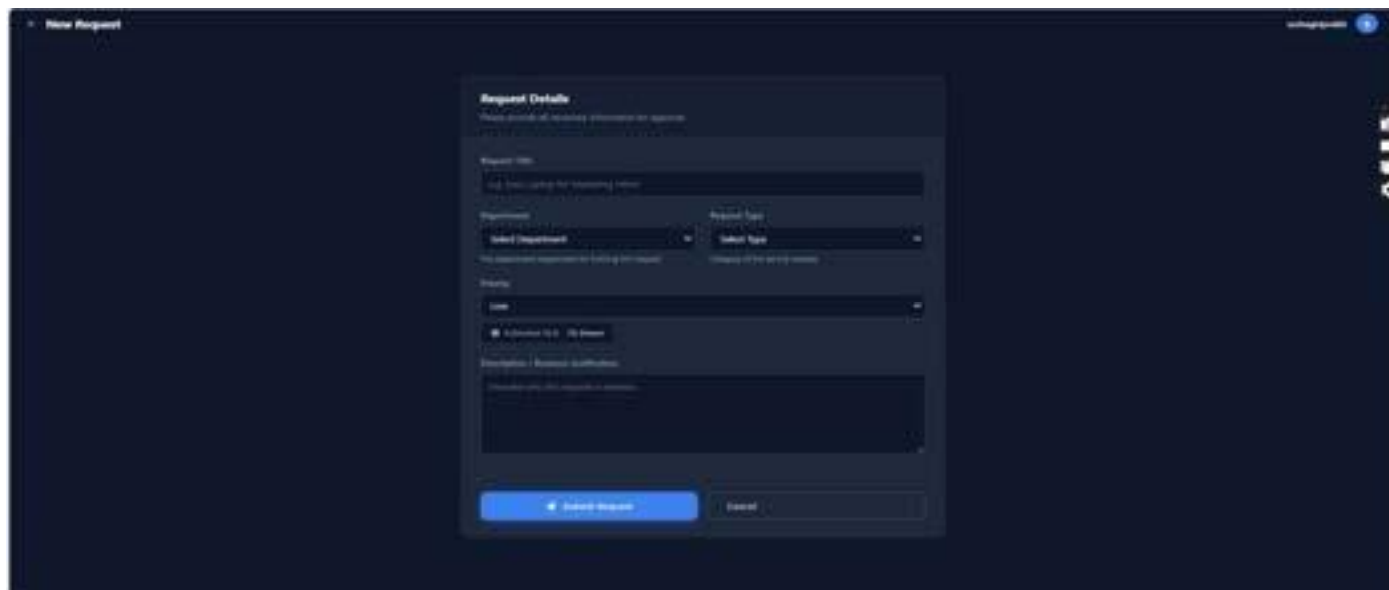


Fig. 5. User Request Submission Interface

C. Performance Analysis

System performance was assessed by measuring API response times for representative operations under simulated concurrent user loads. Load testing was conducted using Apache JMeter with thread pools configured to simulate 10, 25, and 50 concurrent users. The request submission API endpoint achieved mean response times of 87 ms, 134 ms, and 198 ms under 10, 25, and 50 concurrent users, respectively. The approval action endpoint exhibited mean response times of 72 ms, 118 ms, and 183 ms under the same load conditions. The administrative dashboard data retrieval endpoint, which aggregates across all active requests, recorded mean response times of 143 ms, 221 ms, and 347 ms under increasing load.

All measured response times remained below 400 ms under the 50-concurrent-user load, which represents the anticipated peak operational load for the target deployment context. No request failures or database deadlock conditions were observed during load testing. Memory utilization of the Spring Boot process remained stable at approximately 320 MB under sustained load, and MySQL query execution times for indexed lookups remained below 5 ms throughout the testing period. These results confirm that the system delivers adequate performance for its intended organizational deployment scale.

Endpoint	10 users	25 users	50 users	P95 @50
POST /api/requests	91 ms	139 ms	204 ms	318 ms
POST /api/approvals/{id}/action	78 ms	124 ms	191 ms	287 ms



GET /api/admin/requests	151 ms	238 ms	361 ms	489 ms
-------------------------	--------	--------	--------	--------

Table I. API Response Times (mean) at Three Concurrency Levels

D. User Interface Evaluation

The user interface was evaluated through structured walkthroughs conducted with five representative users drawn from each target role (requestor, approver, and administrator). Participants were asked to complete a set of prescribed tasks including submitting a new service request, reviewing and acting on a pending approval, and generating a summary report from the administrative dashboard. Task completion rates, error rates, and qualitative feedback were recorded.

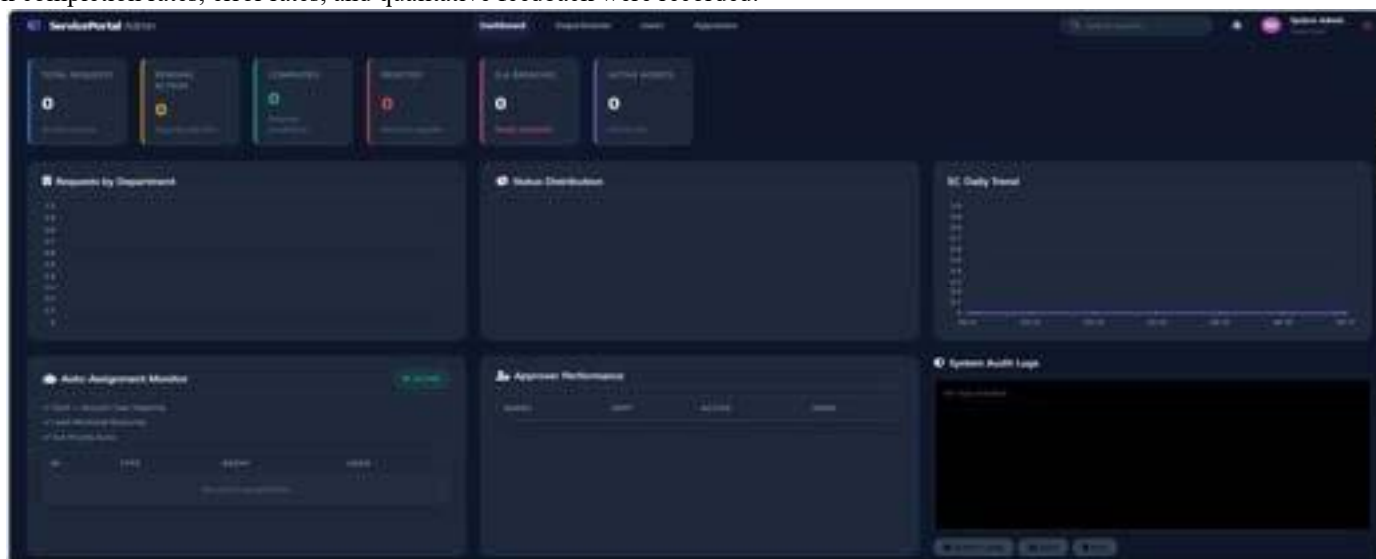


Fig. 6. Administrator Dashboard Interface

All participants successfully completed all assigned tasks without assistance. The mean time to complete the request submission task was 2.3 minutes, with no participant requiring more than 3.5 minutes. Participants rated the interface clarity, navigational structure, and status visibility positively, with all ratings at or above four out of five on a Likert scale. Constructive feedback highlighted the desirability of a mobile-optimized layout and an in-system messaging feature for direct communication between requestors and approvers; these observations are noted as enhancements for future development.

V. DISCUSSION AND LIMITATIONS

The implementation and evaluation outcomes described in Section IV confirm that SRWAS fulfills its stated research objectives and delivers measurable improvements over conventional manual service request handling. The configurable workflow engine successfully accommodates diverse service category requirements without code modification, validating the architectural decision to externalize workflow definitions as data rather than logic.

The BCrypt-based authentication mechanism and stateless JWT session model provide a sound security posture appropriate for organizational deployment over internal networks.

The comparative advantage of SRWAS relative to existing systems is most pronounced in three dimensions. First, the system's opensource technology stack (Spring Boot, MySQL) eliminates licensing costs that would otherwise render commercial ITSM platforms inaccessible to budget-constrained organizations. Second, the modular, layered architecture facilitates extension and customization without imposing the complexity overhead typical of enterprise platform configurations. Third, the RESTful API surface



enables integration with complementary organizational systems—such as LDAP directories for centralized identity management or ERP systems for procurement workflow synchronization—without structural modifications to the SRWAS core.

Notwithstanding these contributions, the current implementation exhibits several limitations that warrant acknowledgment. The prototype has been evaluated under controlled laboratory conditions with a simulated user base, and real-world deployment may surface performance and usability characteristics not captured in the controlled study. The user interface, while functional and wellrated by evaluation participants, has not been optimized for mobile or tablet form factors, which may limit accessibility for fieldbased requestors.

The current notification mechanism relies on synchronous SMTP dispatch within the application thread pool; at elevated request volumes, this approach may introduce processing latency, suggesting the adoption of an asynchronous message queue (e.g., RabbitMQ or Apache Kafka) for notification delivery in high-throughput environments. Additionally, the current implementation does not provide a self-service workflow configuration interface for administrators; workflow rule modifications presently require direct database intervention, which is a barrier for non-technical system managers.

The absence of multi-tenancy support in the current architecture constrains the system to single-organization deployments. Organizations with complex subsidiary structures or federated service management requirements would require architectural extensions to support tenant isolation. Furthermore, the current reporting capability, while adequate for operational monitoring, does not support advanced analytics or predictive insights that could assist administrators in identifying systematic service delivery bottlenecks.

VI. CONCLUSION AND FUTURE WORK

This paper has presented the design, implementation, and evaluation of the Service Request Workflow and Automation System (SRWAS), a web-based platform for automating the management of organizational service requests across IT support, facilities management, human resources, and procurement functions. The system addresses well-documented deficiencies in manual service request handling by providing a structured submission interface, a configurable multi-stage approval workflow engine, role-based access control, real-time status tracking, and automated email notifications. The implementation leverages a Java Spring Boot backend, MySQL database, and HTML/CSS/JavaScript frontend, delivering a deployable and maintainable solution built entirely on opensource technologies.

Functional testing demonstrated complete coverage of specified system behaviors with all test cases passing after minor correction of a boundary condition in the escalation module. Performance evaluation under simulated concurrent load confirmed response times within acceptable operational bounds across all primary API endpoints. User interface evaluation with representative stakeholders yielded consistently positive ratings, with successful task completion across all assigned scenarios.

Future development efforts will pursue several enhancement directions. A graphical workflow configuration interface will be developed to empower non-technical administrators to define and modify approval chains without direct database access. Mobileresponsive design improvements will be applied to the frontend to support access from portable devices. An asynchronous notification dispatch mechanism will replace the current synchronous model to improve throughput under high request volumes. Integration connectors for widely adopted LDAP and Active Directory identity providers will be developed to enable centralized user provisioning. Advanced analytics capabilities, including predictive workload modeling and service-level agreement (SLA) breach forecasting, will be explored as research extensions. Finally, multi-tenancy support will be incorporated to enable the system's deployment as a shared service platform for organizations with federated management structures.

VII. ACKNOWLEDGMENT

The authors wish to express their sincere appreciation to Bonam Venkata Chalamayya Engineering College for providing the academic infrastructure, laboratory resources, and institutional support that enabled the execution of this research. The authors are grateful to the faculty mentors and department supervisors for their expert technical guidance, constructive review of the manuscript, and continuous encouragement throughout the project. The authors also acknowledge the time and feedback contributed by the participants



in the user interface evaluation study. The collective support of colleagues and peers within the department has been invaluable in the refinement of both the system design and the research presentation.

REFERENCES

- [1] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, "Workflow Patterns," *Distributed and Parallel Databases*, vol. 14, no. 1, pp. 5–51, 2003, doi: 10.1023/A:1022883727209.
- [2] M. Marrone and G. Hammerle, "IT Service Management: A Cross-National Study of ITIL Adoption," *Communications of the Association for Information Systems*, vol. 34, no. 1, pp. 865–892, 2014, doi: 10.17705/ICAIS.03447.
- [3] A. Cater-Steel and M. Toleman, "Education for IT Service Management Standards," *International Journal of IT Standards and Standardization Research*, vol. 5, no. 2, pp. 27–41, 2007, doi: 10.4018/jitsr.2007070102.
- [4] M. Weske, *Business Process Management: Concepts, Languages, Architectures*, 3rd ed. Berlin, Germany: Springer, 2019, doi: 10.1007/978-3-662-59432-2.
- [5] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.
- [6] X. Liu, W. Huang, and Y. Zhang, "RESTful API Design Patterns for Enterprise Service Integration," in *Proc. IEEE International Conference on Services Computing (SCC)*, Milan, Italy, 2019, pp. 132–139, doi: 10.1109/SCC.2019.00029.
- [7] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST Standard for Role-Based Access Control," *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 224–274, 2001, doi: 10.1145/501978.501980.
- [8] S. Dustdar and W. Schreiner, "A Survey on Web Services Composition," *International Journal of Web and Grid Services*, vol. 1, no. 1, pp. 1–30, 2005, doi: 10.1504/IJWGS.2005.007545.
- [9] C. Walls, *Spring Boot in Action*. Shelter Island, NY, USA: Manning Publications, 2016.
- [10] S. Ahmed, M. Raza, and F. Iqbal, "Relational Database Optimization Strategies for Service Management Applications," in *Proc. IEEE International Conference on Cloud Engineering (IC2E)*, Prague, Czech Republic, 2021, pp. 55–63, doi: 10.1109/IC2E52221.2021.00017.
- [11] T. Davenport, "The Coming Commoditization of Processes," *Harvard Business Review*, vol. 83, no. 6, pp. 100–108, 2005.
- [12] A. Koschmider, M. Song, and H. A. Reijers, "Advanced Recommendations for Business Process Modeling with the Use of Graph Edit Distance," in *Proc. CAiSE Forum*, Amsterdam, Netherlands, 2010, pp. 41–52.

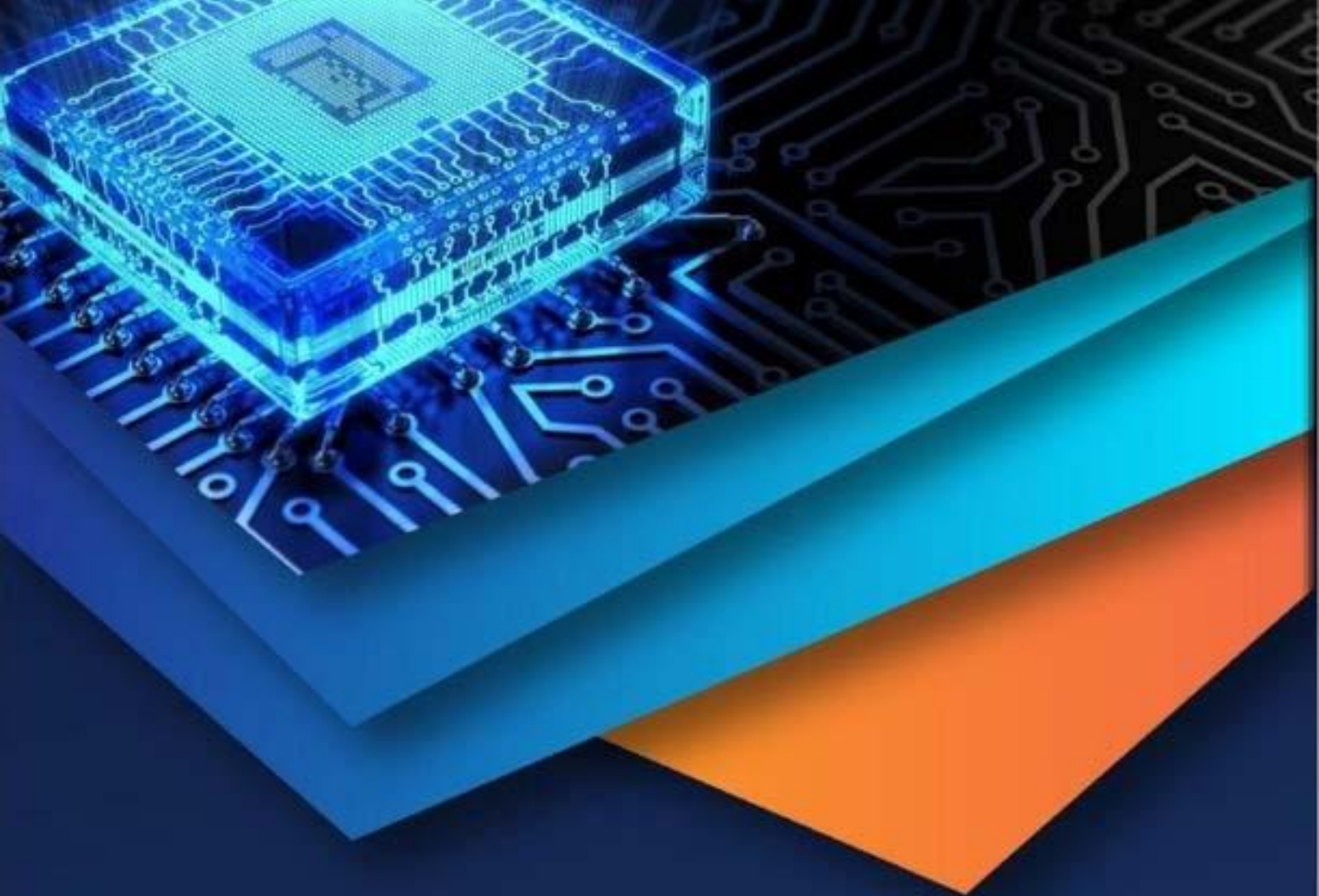


AUTHOR BIOGRAPHIES

	Vanacharla Vijaya Phani Sai Sahithi  Linkedin,  Orcid currently residing on GSR Layout-1, Kamanagaruvu (533213), is a B.Tech student specializing in Artificial Intelligence and Machine Learning at Bonam Venkata Chalamayya Engineering College, Odalarevu, with an expected graduation in April 2026. She aims to secure a position that leverages her strong organizational skills, educational background, and ability to work effectively with others. She possesses key skills in communication, self-motivation, analytical and critical thinking, and dependability. While her professional experience is listed as a student, her proactive approach and skill set indicate a strong potential for growth and contribution in a professional setting. For further contact, she can be reached at 6300814535 or via at sahithi888vanacharla@gmail.com
	Kadiyala Yaswanth Durgendra  Linkedin,  Orcid, currently residing on Puvalla Street, G.kothapalli (533285), is a B.Tech student specializing in Artificial Intelligence and Machine Learning at Bonam Venkata Chalamayya Engineering College, Odalarevu, with an expected graduation in April 2026. He aims to secure a position that leverages his strong organizational skills, educational background, and ability to work effectively with others. possesses key skills in communication, selfmotivation, analytical and critical thinking, and dependability. While his professional experience is listed as a student, his proactive approach and skill set indicate a strong potential for growth and contribution in a professional setting. For further contact, he can be reached at 9550816873 or via email at kadiyalayaswanth123@gmail.com
	Kanuri Jagan Satya Sai  Linkedin,  Orcid, currently residing on Cheyyuru Agraharam, Ramalayam Street, Katrenikona Manadalam (533212), is a B.Tech student specializing in Artificial Intelligence and Machine Learning at Bonam Venkata Chalamayya Engineering College, Odalarevu, with an expected graduation in April 2026. He aims to secure a position that leverages his strong organizational skills, educational background, and ability to work effectively with others. possesses key skills in communication, self-motivation, analytical and critical thinking, and dependability. While his professional experience is listed as a student, his proactive approach and skill set indicate a strong potential for growth and contribution in a professional setting. For further contact, he can be reached at 8688295885 or via email at 23225a6103@bvcgroup.in



	Prakki Mohan Satya Seshagiri  Linkedin  Orcid, currently residing on Ramalakshmi Colony, Janupalli (533201), is a B.Tech student specializing in Artificial Intelligence and Machine Learning at Bonam Venkata Chalamayya Engineering College, Odalarevu, with an expected graduation in April 2026. He aims to secure a position that leverages his strong organizational skills, educational background, and ability to work effectively with others. possesses key skills in communication, self motivation, analytical and critical thinking, and dependability. While his professional experience is listed as a student, his proactive approach and skill set indicate a strong potential for growth and contribution in a professional setting. For further contact, he can be reached at 9989875668 or via email at seshagiriprakki21@gmail.com
	Ganji Lakshmi is an Assistant Professor in the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning) at Bonam Venkata Chalamayya Engineering College, affiliated with JNTU Kakinada, Andhra Pradesh, India. She holds a Master of Technology (M.Tech) degree from BVC Institute of Technology and Science, Batlapalem. She has experience in teaching and mentoring undergraduate students. Her research interests include machine learning, image processing, and data analytics. She can be contacted at ganjilakshmi292.bvce@bvcgroup.in.  Orcid



10.22214/IJRASET



45.98



IMPACT FACTOR:
7.129



IMPACT FACTOR:
7.429



INTERNATIONAL JOURNAL FOR RESEARCH

IN APPLIED SCIENCE & ENGINEERING TECHNOLOGY

Call : 08813907089  (24*7 Support on Whatsapp)



ISSN No. : 2321-9653

IJRASET

**International Journal for Research in Applied
Science & Engineering Technology**

IJRASET is indexed with Crossref for DOI-DOI : 10.22214

Website : www.ijraset.com, E-mail : ijraset@gmail.com

Certificate

*It is here by certified that the paper ID : IJRASET78451, entitled
SRWAS: A WebBased Service Request and Workflow Automation System
by
Kanuri Jagan Satya Sai*

*after review is found suitable and has been published in
Volume 14, Issue III, March 2026
in*

*International Journal for Research in Applied Science &
Engineering Technology*

By [Signature]

Editor in Chief, IJRASET

ISRA
JIF

ISRA Journal Impact
Factor: 7.429



45.98

INDEX COPERNICUS



THOMSON REUTERS
Researcher ID: N-9681-2016



10.22214/IJRASET



TOGETHER WE REACH THE GOAL
SJIF 7.429